

人工智能数学基础

最优化导论

机器学习中的优化方法

刘志磊

天津大学

智能与计算学部

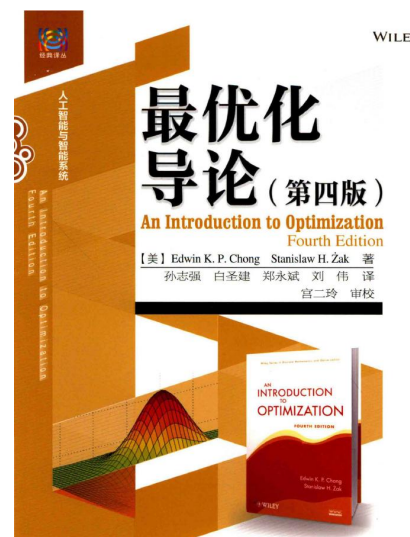
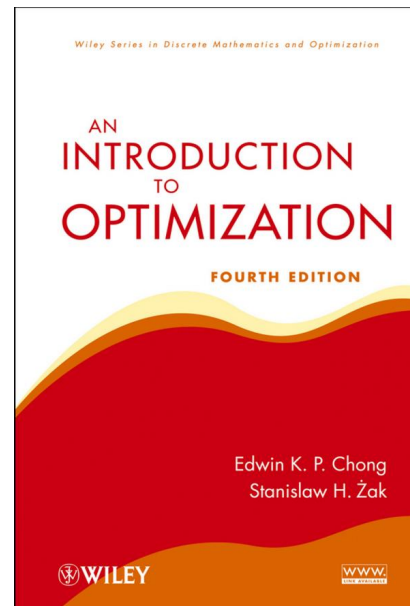
Email: zhileiliu@tju.edu.cn



人工智能学院
School of Artificial Intelligence

最优化导论

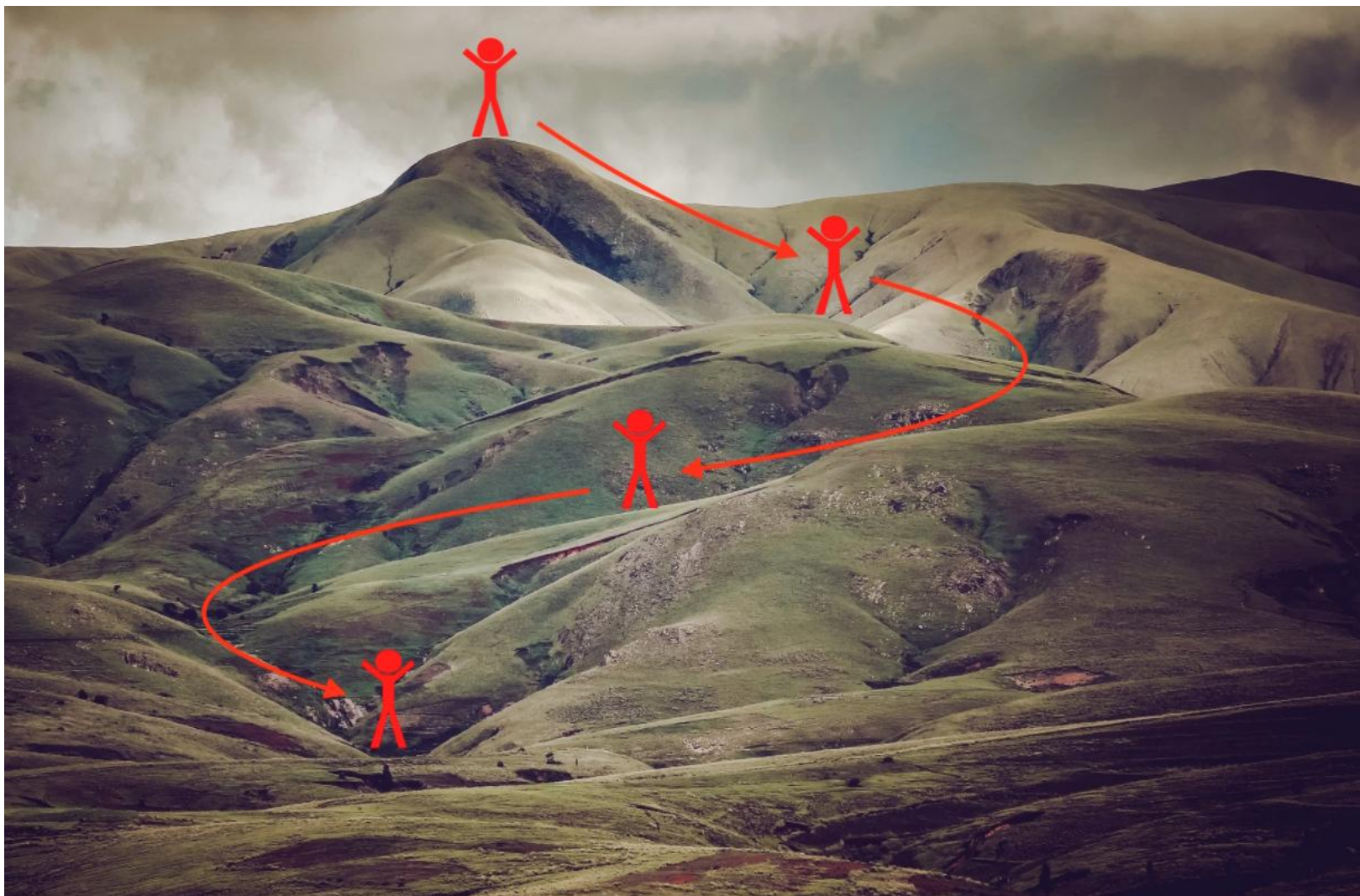
- 1. 最优化问题概述
 - 最优化问题的定义及应用场景
- 2. 最优化数学基础
 - 方向导数、梯度、凸函数、凸集、矩阵分析
- 3. 标准形式和对偶形式
 - 原问题及对偶问题
- 4. 连续优化
 - 最小二乘法、线性回归方程
 - 一阶优化（梯度下降/共轭梯度法）
 - 二阶优化（牛顿法/拟牛顿法/DFP法）
- 5. 图论与离散优化
 - 最短路径问题/TSP问题/最小割最大流



参考教材：
[1] 最优化导论（第四版）电子工业出版社



1. 最优化问题概述



1. 最优化问题概述

- 1.0 优化

- **Optimization** (Oxford dictionary): The action of making the best or most effective use of a situation or resource.

- **Mathematical optimization** or mathematical programming (in mathematics, computer science and operations research) is:

- the selection of a best element from some set of available alternatives

- with regard to some criterion

- **Engineering optimization** uses optimization techniques to aid the engineering design process

资源有限、目标明确、方案可选



1. 最优化问题概述

• 1.0 食谱问题

问题：假设小明喜欢吃的水果有苹果，橘子，香蕉，每种水果分别含有不同量的维生素B和维生素C，如下表。正常人每天需要摄入 $40mg$ 的 V_c 和 $50mg$ 的 V_b ，试确定在保证营养的要求下最经济的食谱。

维生素	苹果	橘子	香蕉	每日需求
V_c (mg)	8	6	4	40
V_b (mg)	10	9	12	50
单价	8	5	4	

假设小明每天摄入三种水果的数量为 x_1 , x_2 , x_3 。食谱问题可以写成如下数学形式：

$$\min \quad 8x_1 + 5x_2 + 4x_3$$

$$s. t. \quad 8x_1 + 6x_2 + 4x_3 \geq 40$$

$$10x_1 + 9x_2 + 12x_3 \geq 50$$

$$x_1, x_2, x_3 \geq 0$$

线性规划



1. 最优化问题概述

• 1.0 资金使用问题

问题：假设有 400 万元资金，4 年内使用完，若一年使用资金 x 万元，可得到利润 $\sqrt{x}$ ，不用的资金可以存入银行，年利率 10% 试制定出获益最多的资金使用规划。

	第一年	第二年	第三年	第四年
现有资金	400	$400 - x_1$	$400 - x_1 - x_2$	$400 - x_1 - x_2 - x_3$
使用资金	x_1	x_2	x_3	x_4

假设每年的资金使用数量为 x_1, x_2, x_3, x_4 。资金使用问题可以写成如下数学形式：

$$\text{第一年} \quad 0 \leq x_1 \leq 400$$

$$\text{第二年} \quad 0 \leq x_2 \leq (400 - x_1) * 1.1$$

$$\text{第三年} \quad 0 \leq x_3 \leq [(400 - x_1) * 1.1 - x_2] * 1.1$$

$$\text{第四年} \quad 0 \leq x_4 = \{[(400 - x_1) * 1.1 - x_2] - x_3\} * 1.1$$

非线性规划



1. 最优化问题概述

• 1.0 资金使用问题

问题：假设有 400 万元资金，4 年内使用完，若一年使用资金 x 万元，可得到利润 $\sqrt{x}$ ，不用的资金可以存入银行，年利率 10% 试制定出获益最多的资金使用规划。

	第一年	第二年	第三年	第四年
现有资金	400	$400 - x_1$	$400 - x_1 - x_2$	$400 - x_1 - x_2 - x_3$
使用资金	x_1	x_2	x_3	x_4

数学模型：

$$\begin{aligned} \max \quad & \sqrt{x_1} + \sqrt{x_2} + \sqrt{x_3} + \sqrt{x_4} \\ \text{s. t.} \quad & x_1 \leq 400 \\ & 1.1x_1 + x_2 \leq 400 \\ & 1.21x_1 + 1.1x_2 + x_3 \leq 484 \\ & 1.331x_1 + 1.21x_2 + 1.1x_3 + x_4 \leq 532.4 \\ & x_1, x_2, x_3, x_4 \geq 0 \end{aligned}$$



1. 最优化问题概述

• 1.1 最优化

- **定义：**研究某些数学上定义问题的最优解，即对给出的实际问题，在众多方案中选出**最优方案**。

$$\underline{\min \quad 8x_1 + 5x_2 + 4x_3} \quad \text{目标函数}$$

$$\begin{array}{l} s. t. \quad 8x_1 + 6x_2 + 4x_3 \geq 40 \\ \quad \quad 10x_1 + 9x_2 + 12x_3 \geq 50 \\ \quad \quad \quad x_1, x_2, x_3 \geq 0 \end{array} \quad \left. \vphantom{\begin{array}{l} s. t. \\ \quad \quad 10x_1 + 9x_2 + 12x_3 \geq 50 \\ \quad \quad \quad x_1, x_2, x_3 \geq 0 \end{array}} \right\} \text{约束条件}$$

- **约束条件：**反映有限资源对生产经营活动的种种约束，或者生产经营必须完成的任务
- **目标函数：**反映生产经营者在有限资源条件下希望达到的生产或经营的目标

1. 最优化问题概述

• 1.1 最优化

- 最优化问题的一般形式

$$\min \quad f(x) \quad \text{目标函数}$$

$$s. t. \quad h_i(x) = 0, i = 1, 2, \dots, m \quad \text{等式约束}$$

$$g_j(x) \geq 0, j = 1, 2, \dots, p \quad \text{不等式约束}$$

一般 x 是 n 维向量，在实际应用中，可将求最大值的目标函数取相反数后统一成公式中的求最小值的形式。

- 基本术语

$x \in \mathbb{R}^n$ – optimization variable (vector with n real-valued entries)

$f: \mathbb{R}^n \rightarrow \mathbb{R}$ – optimization objective

$\mathcal{C} \subseteq \mathbb{R}^n$ – constraint set

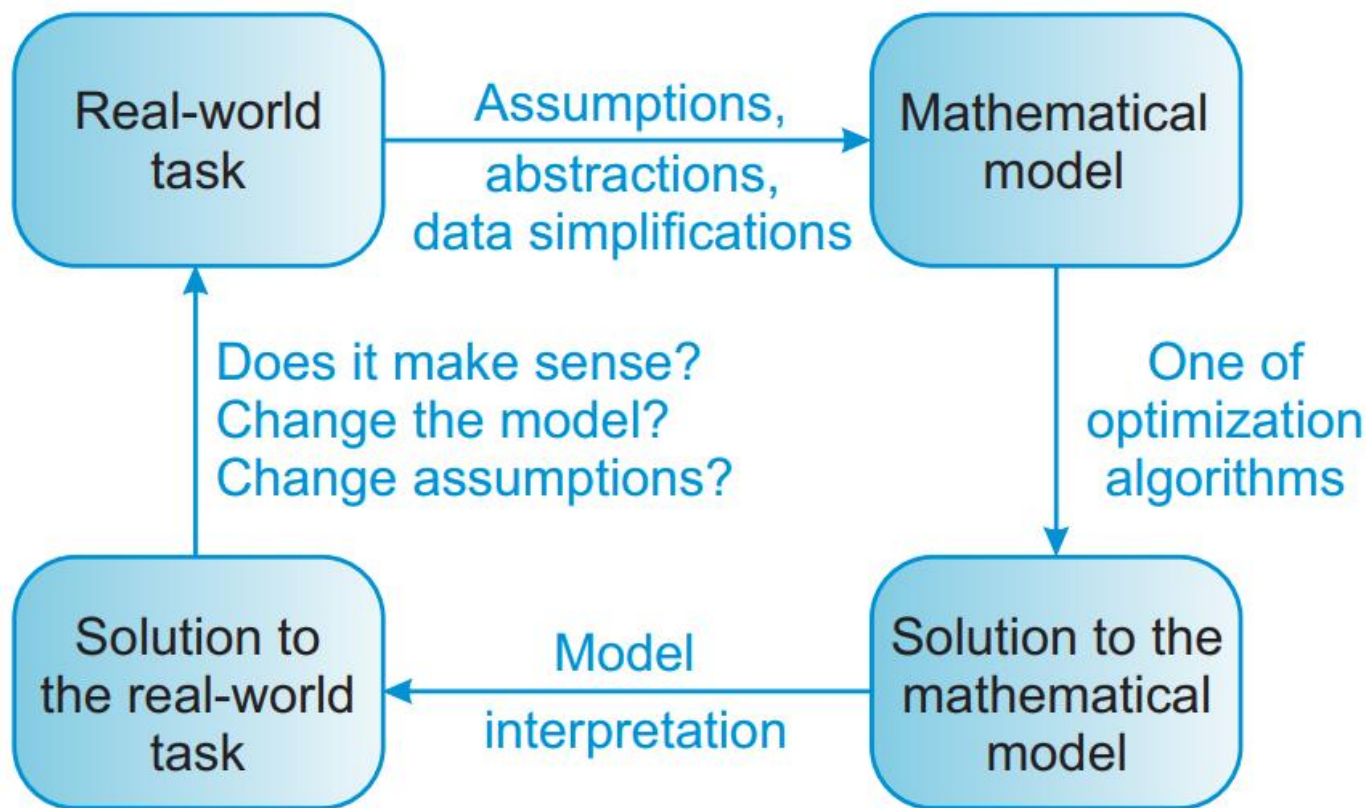
$x^* \equiv \operatorname{argmin}_{x \in \mathcal{C}} f(x)$ – optimal solution

$f^* \equiv f(x^*) \equiv \min_{x \in \mathcal{C}} f(x)$ – optimal objective



1. 最优化问题概述

- 1.1 最优化

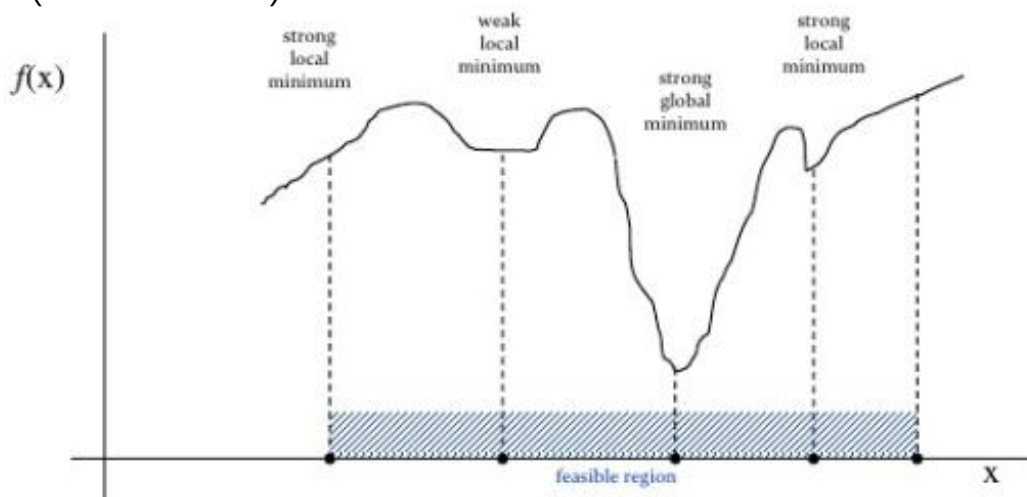
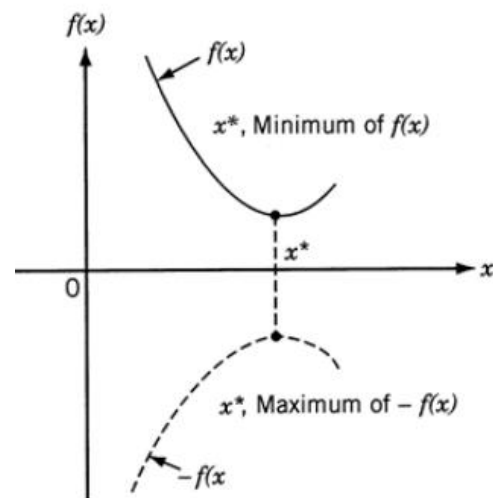


1. 最优化问题概述

• 1.1 最优化

• Step 1: 建立模型

- 建模 (Modeling) : 问题定义
 - 目标 (Objective)
 - 变量 (Variables or the unknowns)
 - 约束 (Constraints)



1. 最优化问题概述

- 1.1 最优化

- Step 2: 确定问题类型, 选择合适的优化方法

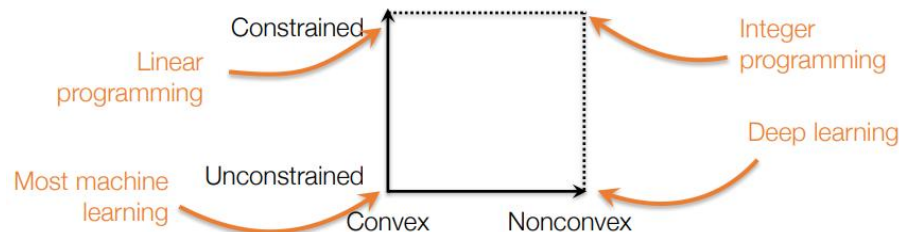
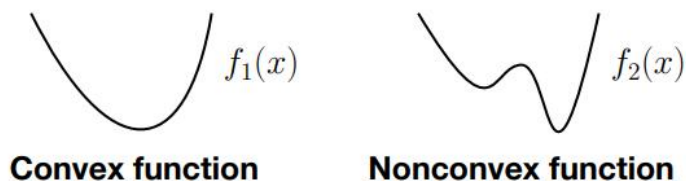
- Continuous vs. discrete
- Static (=optimizes parameters) vs. dynamic (=optimizes shape of the function)
- Unconstrained vs. constrained optimization
- Deterministic optimization vs. optimization under uncertainty
- None, one objective or many objectives.



1. 最优化问题概述

• 1.2 最优化问题分类

• 按目标函数性质分类



类型	定义	机器学习场景	求解方法
无约束优化	无约束条件，直接优化目标函数 $\min f(x)$ 。	神经网络训练（梯度下降系列算法）	梯度下降、牛顿法等
约束优化	存在等式或不等式约束，如 $\min f(x) \text{ s.t. } g(x) \leq 0$ 。	支持向量机（最大化间隔 + 样本约束）	拉格朗日乘数法、KKT 条件等
凸优化	目标函数为凸函数，可行域为凸集，局部最优即全局最优。	线性回归（平方损失为凸函数）	凸优化算法（如内点法）
非凸优化	目标函数为非凸函数（如存在多个局部极小值）。	深度神经网络（非凸损失函数）	随机梯度下降（SGD）、启发式算法

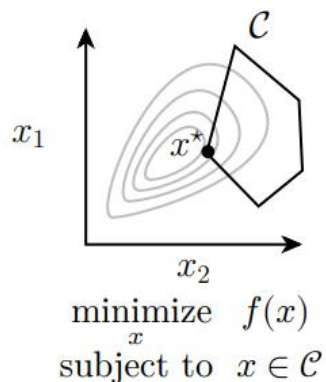
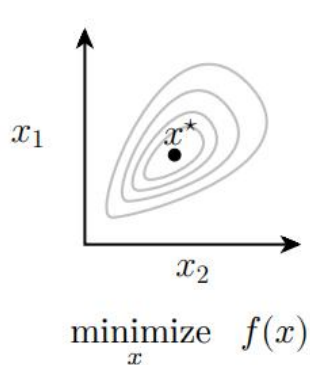
1. 最优化问题概述

• 1.2 最优化问题分类

• 最优化问题

- 约束优化和无约束优化 (constrained vs. unconstrained)

Constrained vs. unconstrained



1. 最优化问题概述

• 1.2 最优化问题分类

• 按变量类型分类

- **连续优化**：决策变量 x 取值为连续空间（如实数域）
 - 例：神经网络参数
- **离散优化**：决策变量 x 取值为离散集合（如整数、有限集合）
 - 例：特征选择（变量为 0/1，表示是否选择特征）、组合优化（如旅行商问题）
 - 难点：离散空间难以用梯度等连续数学工具求解，需借助启发式算法（如遗传算法、模拟退火）

• 按优化目标数量分类

- **单目标优化**：优化单个目标函数（如最小化损失函数）。
- **多目标优化**：同时优化多个冲突目标（如模型准确率与复杂度、计算速度与精度）。



1. 最优化问题概述

• 1.2 最优化问题分类

• 按数据与优化过程的关系分类

- **批量优化 (Batch Optimization)** : 使用全部数据计算梯度 (如批量梯度下降) 。
 - 优点: 稳定性高, 收敛到全局最优 (若目标函数凸) 。
 - 缺点: 数据量大时计算成本高。
- **随机优化 (Stochastic Optimization)** : 使用随机样本 (mini-batch 或单个样本) 近似梯度 (如随机梯度下降 SGD) 。
 - 优点: 计算效率高, 适合大规模数据。
 - 缺点: 存在随机性, 可能陷入局部最优或震荡。
- **在线优化 (Online Optimization)** : 数据按序列逐个或逐批到达, 实时更新模型参数 (如在线学习) 。
 - 例: 推荐系统实时反馈优化。



1. 最优化问题概述

- 1.2 最优化问题分类

- 按优化算法的理论基础分类

- 基于导数的方法：利用目标函数的梯度（一阶导数）或海森矩阵（二阶导数）引导优化

- 例：梯度下降、牛顿法、拟牛顿法（如 L-BFGS）。

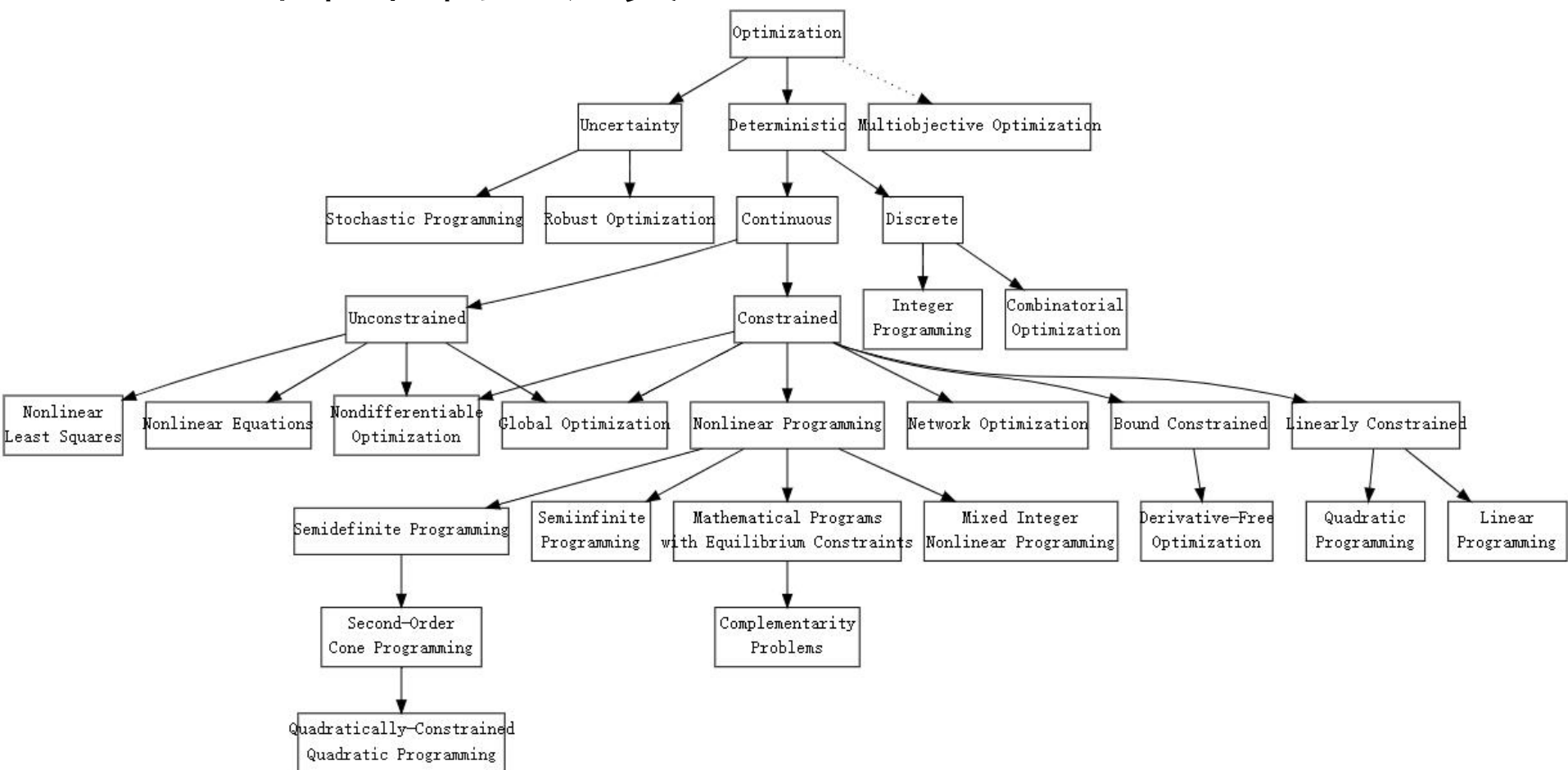
- 无导数优化（**Derivative-Free Optimization**）：不依赖导数信息，通过函数值比较或采样搜索最优解

- 例：网格搜索、贝叶斯优化（用于超参数调优）、粒子群优化（PSO）。



1. 最优化问题概述

• 1.2 最优化问题分类



[Optimization Taxonomy | NEOS \(neos-guide.org\)](https://neos-guide.org/)



1. 最优化问题概述

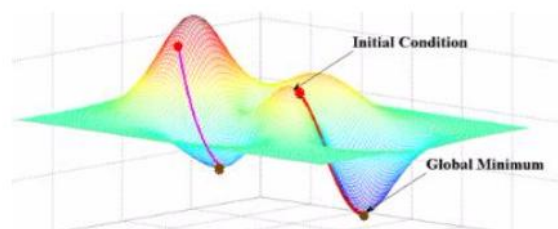
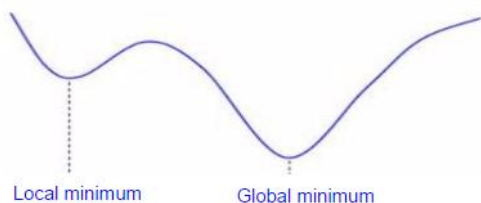
• 1.3 应用场景

应用场景分类	核心目标	典型案例	关键优化方法 / 技术
监督学习	最小化预测损失（分类 / 回归误差）	图像分类（ResNet 交叉熵损失）、房价预测（线性回归 MSE）	随机梯度下降（SGD）、Adam、最小二乘法
无监督学习	挖掘数据内在结构（聚类、降维）	K-means 聚类、PCA 降维、深度自编码器	K-means 迭代优化、SVD 分解、反向传播 + 梯度下降
强化学习	最大化长期累积奖励（策略优化）	Atari 游戏 AI（DQN）、AlphaGo Zero	时序差分（TD）学习、策略梯度（PPO）、MCTS
生成对抗网络	通过对抗博弈逼近真实数据分布	图像生成（StyleGAN）、文本生成（TextGAN）	极小极大博弈优化、梯度惩罚（WGAN-GP）
超参数调优	在离散空间中搜索最优超参数组合	SVM 调优（搜索 C 和 γ ）、神经网络架构搜索（NAS）	网格搜索、贝叶斯优化、遗传算法
模型压缩与加速	在精度约束下最小化模型参数 / 计算量	神经网络剪枝（L1 正则化）、量化（8 位整数参数）、知识蒸馏	稀疏优化、离散优化、KL 散度约束优化
多目标优化	平衡多个冲突目标（如准确率、速度、隐私）	神经网络架构搜索（NAS）、联邦学习（精度 + 隐私保护）	帕累托优化、带约束的凸优化（如差分隐私）
在线学习与流式数据	实时更新模型以适应动态数据分布	推荐系统实时优化、欺诈检测	在线梯度下降（OGD）、自适应学习率（Adagrad）
分布式与并行优化	加速大规模数据 / 复杂模型训练（如大模型）	GPT-4 分布式训练、GloVe 词向量数据并行训练	数据并行（参数服务器）、模型并行、异步优化



1. 最优化问题概述

• 1.3 应用场景



- **整体最优解**: 若 $x^* \in D$, 对于一切 $x \in D$ 恒有 $f(x^*) \leq f(x)$, 则称 x^* 为最优化问题的整体最优解; 若 $x^* \in D$, 对于一切 $x \in D$ 恒有 $f(x^*) < f(x)$, 则称 x^* 为最优化问题的严格整体最优解;
- **局部最优解**: 若 $x^* \in D$, 存在某一个邻域 $N_\varepsilon(x^*)$, 对于一切 $x \in N_\varepsilon(x^*)$ 恒有 $f(x^*) \leq f(x)$, 则称 x^* 为最优化问题的局部最优解; 若 恒有 $f(x^*) < f(x)$, 则称 x^* 为最优化问题的严格局部最优解;

1. 最优化问题概述

- 1.3 应用场景

- Example: image deblurring



(a) Original image.



(b) Blurry, noisy image.



(c) Restored image.

Figure from (O'Connor and Vandenberghe, 2014)

- 给定有损图像 $Y \in \mathbb{R}^{m \times n}$ ，则可以通过解决以下优化问题实现图像修复：

$$\underset{X}{\text{minimize}} \sum_{i,j} |Y_{ij} - (K * X)_{ij}| + \lambda \sum_{i,j} \left((X_{ij} - X_{i,j+1})^2 + (X_{i+1,j} - X_{ij})^2 \right)^{\frac{1}{2}}$$

- $K*$ 表示模糊滤波卷积操作

1. 最优化问题概述

• 1.3 应用场景

• Example: robot trajectory planning

- 许多机器人规划任务比最短路径更加复杂，例如包括机器人动力学的任务，需要“平滑”控制。
- 常见的做法是把规划问题看作一个优化任务进行建模。

Robot state x_t and control inputs u_t

$$\begin{aligned} & \underset{x_{1:T}, u_{1:T-1}}{\text{minimize}} && \sum_{i=1}^T \|u_t\|_2^2 \\ & \text{subject to} && x_{t+1} = f_{\text{dynamics}}(x_t, u_t) \\ & && x_t \in \text{FreeSpace}, \forall t \\ & && x_1 = x_{\text{init}}, x_T = x_{\text{goal}} \end{aligned}$$

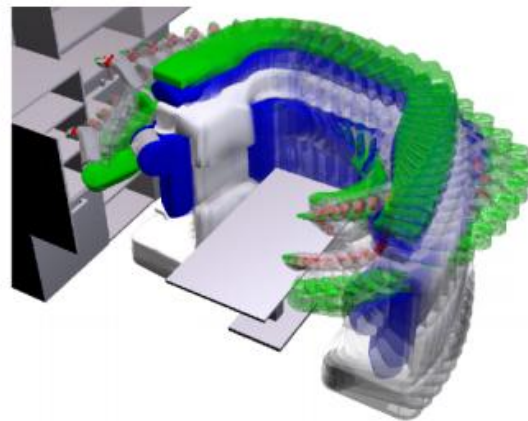


Figure from (Schulman et al., 2014)

1. 最优化问题概述

- 1.3 应用场景

- Example: Machine Learning

- 几乎所有的有监督机器学习算法的训练是在解决以下最优化问题

$$\underset{\theta}{\text{minimize}} \sum_{i=1}^m \ell(h_{\theta}(x^{(i)}), y^{(i)})$$

Where $x^{(i)} \in \mathcal{X}$ are inputs, $y^{(i)} \in \mathcal{Y}$ are outputs, ℓ is a loss function, and h_{θ} is a hypothesis function parameterized by θ , which are the parameters of the model we are optimizing over

$$\min_w \frac{1}{N} \sum_{i=1}^N L(w, x_i, y_i) + \lambda \|w\|_2^2 \qquad \min_s \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu_i\|^2$$



2. 最优化数学基础

数学领域	核心作用	对应优化场景
微积分与梯度	定义优化方向（如梯度下降）、判断极值条件	监督学习损失最小化、无约束优化
凸分析	保证全局最优解的存在性与高效求解（如线性规划、SVM）	凸问题（如正则化损失函数）
矩阵理论	分析算法收敛性、处理高维数据（如SVD降维）	特征工程、分布式优化
概率统计	建模数据不确定性、设计贝叶斯优化算法	超参数调优、在线学习



2. 最优化数学基础

- 方向导数
- 梯度
- Hessian 矩阵
- 凸函数、凸集、凸优化
- ~~向量、矩阵、范数~~
- ~~线性/非线性规划、图优化、整数规划...~~
- ~~概率论~~

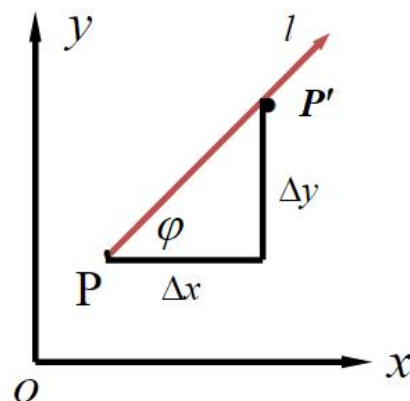


2. 最优化数学基础

• 2.1 方向导数

定义：设函数在点 p 处引一条射线 l , 当 p_0 趋近与点 p 时, 下列极限如果存在, 称为方向导数

$$\frac{\partial f}{\partial l} = \lim_{p \rightarrow p_0} \frac{f(p) - f(p_0)}{|pp_0|}$$



- 方向导数表示一个函数在某一点沿着某个方向的变化率, 比如在某个点 x 沿着 y 轴正方向的方向导数表示函数在该点在 y 轴正方向上的变化率。
- 方向导数的值可以为正、负、零, 正表示函数在该点沿着该方向增加, 负表示函数在该点沿着该方向减少, 零表示函数在该点沿着该方向没有变化。
- 方向导数的计算相当于对函数求偏导数, 并沿着指定的向量做点积。
- 在 n 维空间中, 一个向量的方向可以用一个单位向量来表示, 方向导数的计算可以推广到任意的 n 维向量中。

2. 最优化数学基础

• 2.1 方向导数

- **定理：** 若函数 $z = f(x, y)$ 在点 $p(x, y)$ 处可微，则函数在该点任意方向 l 的方向导数都存在。

$$\frac{\partial f}{\partial l} = \frac{\partial f}{\partial x} \cos \varphi + \frac{\partial f}{\partial y} \cos \beta$$

$\cos \alpha, \cos \beta$ 是方向 l 的方向余弦。拓展到三维：

$$\frac{\partial f}{\partial l} = \frac{\partial f}{\partial x} \cos \varphi + \frac{\partial f}{\partial y} \cos \beta + \frac{\partial f}{\partial z} \cos \gamma$$



2. 最优化数学基础

• 2.1 方向导数

- **例1:** 若函数 $z = x^3 y^2$, 定点 $p_0(3,1), p_1(2,3)$, 求函数在 p_0 沿 $\overrightarrow{p_0 p_1}$ 方向的方向导数。

- **解:**
$$\left. \frac{\partial z}{\partial x} \right|_{p_0} = 3x^2 y^2 \Big|_{p_0} = 27, \quad \left. \frac{\partial z}{\partial y} \right|_{p_0} = 2x^3 y \Big|_{p_0} = 54$$

$$\overrightarrow{P_0 P_1} = (-1, 2), \quad |\overrightarrow{P_0 P_1}| = \sqrt{5}$$

$$\cos \alpha = \frac{-1}{\sqrt{5}}, \quad \cos \beta = \frac{2}{\sqrt{5}}$$

$$\left. \frac{\partial z}{\partial l} \right|_{p_0} = 27 \cdot \left(-\frac{1}{\sqrt{5}} \right) + 54 \cdot \frac{2}{\sqrt{5}} = \frac{81}{\sqrt{5}}$$



2. 最优化数学基础

• 2.1 方向导数

• **问题：** 函数 $z = f(x, y)$ 沿什么方向的方向导数最大或最小？

• **解：**

$$\frac{\partial f}{\partial l} = \frac{\partial f}{\partial x} \cos \varphi + \frac{\partial f}{\partial y} \cos \beta = \vec{G} \vec{l}$$

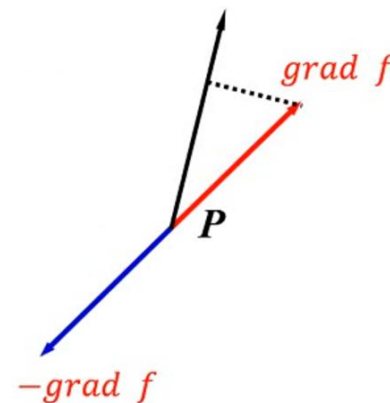
$$\vec{G} = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right] \quad \vec{l} = (\cos \varphi, \cos \beta)$$

$$\frac{\partial f}{\partial l} = |\vec{G}| |\vec{l}| \cos(\vec{G}, \vec{l}) = |\vec{G}| \cos(\vec{G}, \vec{l})$$

当 $\vec{G}$ 与 $\vec{l}$ 方向一致时，方向导数最大；
当 $\vec{G}$ 与 $\vec{l}$ 方向相反时，方向导数最小。

2. 最优化数学基础

• 2.2 梯度 (Gradient)



- **定义：** 以 $f(X)$ 的 n 个偏导数为分量的向量称为 $f(X)$ 在 X 处的梯度，记为：

$$\nabla f(X) = \left(\frac{\partial f(X)}{\partial x_1}, \frac{\partial f(X)}{\partial x_2}, \dots, \frac{\partial f(X)}{\partial x_n} \right)^T$$

或记为 $\text{grad } f(X)$

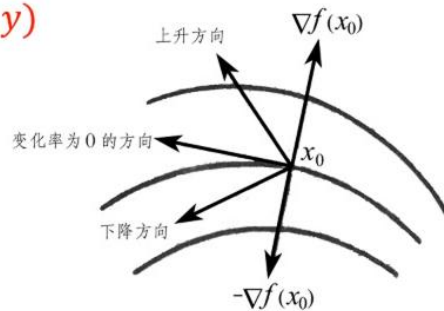
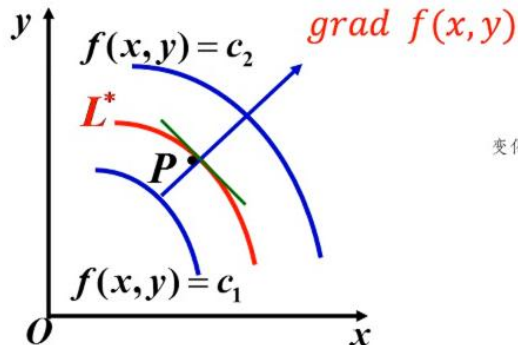
- 梯度是一个向量，表示函数在某一点的变化率最快的方向，它的方向和取得最大梯度方向一致，它的模为方向导数的最大值，沿着 $[-\frac{\partial f}{\partial x}, -\frac{\partial f}{\partial y}]$ 方向，函数下降最快。

- 在优化问题中，梯度通常用于指导优化算法的搜索方向，通过不断沿着负梯度方向进行更新，可以逐步接近函数的最小值点。

2. 最优化数学基础

• 2.2 梯度 (Gradient)

• 几何意义



- 假设函数 $z = f(x, y)$ 表示一个曲面，被平面 $z = c$ 所截，称为曲面的等高线，则梯度 $[-\frac{\partial f}{\partial x}, -\frac{\partial f}{\partial y}]$ 为等高线上 p 点的法向量。
- 梯度方向表示函数上升最快的方向。梯度的负方向是函数值下降最快的方向。梯度的模表示在该方向上函数变化率的大小。

2. 最优化数学基础

• 2.2 梯度 (Gradient)

- **例2:** 一个登山者在山坡上点 $(-\frac{3}{2}, -1, -\frac{3}{4})$ 处, 山坡的高度 z 近似为 $z = 5 - x^2 - 2y^2$, 若以 x 轴方向为正南方向, 则他应当怎样往上登才能攀登地最快?

- **解:** $f(x, y) = 5 - x^2 - 2y^2$ 在点 $P(-\frac{3}{2}, -1)$ 的梯度为:

$$f'_x(P) = -2x = 3; \quad f'_y(P) = -4y = 4$$

$$\text{grad } f(P) = (3, 4)$$

当与梯度方向一致时, 攀登最快, 因此方向为:

$$\vec{l} = (\cos \alpha, \cos \beta) = (\frac{3}{5}, \frac{4}{5})$$



2. 最优化数学基础

• 2.2 梯度 (Gradient)

• 方向导数与梯度的关系

- 方向导数是函数某一点在某个方向上的变化率，而梯度是函数本身在某一点上变化率最大的方向；
- 方向导数实际上是梯度向量与某个方向向量的标量乘积。
- 梯度向量的方向是函数增长最快的方向，因此在这个方向上的方向导数也是最大的。

• 梯度在优化问题中的应用

- 梯度下降法 (*Gradient Descent*)：一阶优化迭代算法，在每次迭代中，算法沿着负梯度方向更新解，因为负梯度方向是函数值减少最快的方向；
- 牛顿法 (*Newton's Method*)：二阶优化算法，不仅使用梯度（一阶导数）还使用黑塞矩阵（二阶导数）来寻找函数的局部极值点；
- 共轭梯度法 (*Conjugate Gradient Method*)：这是一种解决大规模稀疏系统线性方程组的算法，也用于最小化二次函数。共轭梯度法利用梯度和前一次迭代的残差信息来构造一组共轭向量，并沿着这些向量进行搜索；
- 梯度在图像处理中扮演着重要的角色，因为它与图像中亮度变化的幅度和方向紧密相关。在图像中，梯度可以表示为图像的一阶导数，通常用来检测边缘、纹理、阴影等特征，如Canny边缘检测器、Sobel算子、光流估计等。



2. 最优化数学基础

• 2.3 Hessian 矩阵（黑塞矩阵）

- **定义：** 设函数 $f(\mathbf{x})$ 在点 $\mathbf{x}$ 处对于自变量的各个分量二阶偏导数都存在，则称函数在该点处二阶可导，称以下矩阵为Hessian矩阵。

$$H(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- **Hessian 矩阵**是一个多元函数的二阶偏导数构成的方阵，描述了函数的局部曲率

参考资料: [Reducing Motion Blur in Ghost Imaging Via the Hessian Matrix](#)



2. 最优化数学基础

• 2.3 Hessian 矩阵（黑塞矩阵）

• 黑塞矩阵在最优化问题中的应用：

- 在最优化中，黑塞矩阵是非常重要的概念，它可以帮助我们确定优化算法的收敛速度和稳定性，如在牛顿法中，黑塞矩阵用于计算搜索方向，它结合了梯度（一阶导数）和黑塞矩阵（二阶导数）来加速收敛到最优值。
- 二次优化问题：对于二次函数，黑塞矩阵是问题的关键部分，因为它完全描述了函数的局部行为。在二次规划中，黑塞矩阵的正定性或负定性决定了问题的凸性。
- 黑塞矩阵还可以帮助我们进行局部极值点的分类和判断。
 - 若黑塞矩阵是正定的，则表示当前点为局部极小值点；
 - 若黑塞矩阵是负定的，则表示当前点为局部极大值点；
 - 若黑塞矩阵不定，则当前点为鞍点。
- 在图像处理中，特别是在三维表面重建中，黑塞矩阵可以用来分析图像的局部曲率，从而帮助重建出物体的表面。



2. 最优化数学基础

• 2.3 Hessian 矩阵（黑塞矩阵）

- **例：**求 $f(x_1, x_2) = 5x_1 + 8x_2 + x_1x_2 - x_1^2 - 2x_2^2$ 的 Hessian 矩阵。

解： $Df(\mathbf{x}) = (\nabla f(\mathbf{x}))^T = \left[\frac{\partial f}{\partial x_1}(\mathbf{x}), \frac{\partial f}{\partial x_2}(\mathbf{x}) \right] = [5 + x_2 - 2x_1, 8 + x_1 - 4x_2]$

Hessian矩阵即二阶导数阵为：

$$H(\mathbf{x}) = D^2f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2}(\mathbf{x}) & \frac{\partial^2 f}{\partial x_2 \partial x_1}(\mathbf{x}) \\ \frac{\partial^2 f}{\partial x_1 \partial x_2}(\mathbf{x}) & \frac{\partial^2 f}{\partial x_2^2}(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} -2 & 1 \\ 1 & -4 \end{bmatrix}$$

2. 最优化数学基础

• 2.3 Hessian 矩阵（黑塞矩阵）

- **例：**求函数 $f(\mathbf{X}) = x_1^4 + 2x_2^3 + 3x_3^2 - x_1^2x_2 + 4x_2x_3 - x_1x_3^2$ 的梯度和Hessian 矩阵。

- 解：

$$\frac{\partial f}{\partial x_1} = 4x_1^3 - 2x_1x_2 - x_3^2$$

$$\frac{\partial f}{\partial x_2} = 6x_2^2 - x_1^2 + 4x_3 \quad \Rightarrow \quad \nabla f(\mathbf{X}) = \begin{bmatrix} 4x_1^3 & - & 2x_1x_2 & - & x_3^2 \\ 6x_2^2 & - & x_1^2 & + & 4x_3 \\ 6x_3 & + & 4x_2 & - & 2x_1x_3 \end{bmatrix}$$

$$\frac{\partial f}{\partial x_3} = 6x_3 + 4x_2 - 2x_1x_3$$

$$\frac{\partial^2 f}{\partial x_1^2} = 12x_1^2 - 2x_2, \quad \frac{\partial^2 f}{\partial x_1x_2} = -2x_1, \quad \frac{\partial^2 f}{\partial x_1x_3} = -2x_3,$$

$$\frac{\partial^2 f}{\partial x_2^2} = 12x_2, \quad \frac{\partial^2 f}{\partial x_2x_3} = 4, \quad \frac{\partial^2 f}{\partial x_3^2} = 6 - 2x_1,$$



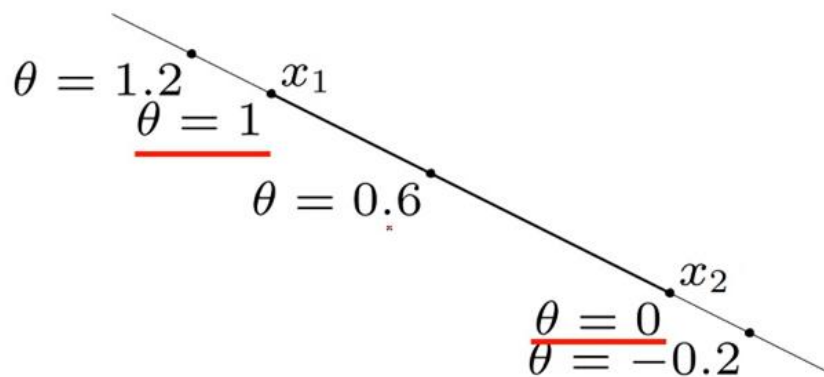
$$\nabla^2 f(\mathbf{X}) = \begin{bmatrix} 12x_1^2 - 2x_2 & -2x_1 & -2x_3 \\ -2x_1 & 12x_2 & 4 \\ -2x_3 & 4 & 6 - 2x_1 \end{bmatrix}$$



2. 最优化数学基础

• 2.4 凸集

- **线性组合**：经过 x_1 和 x_2 直线上的任意一点，均可表示为 x_1 和 x_2 的线性组合。



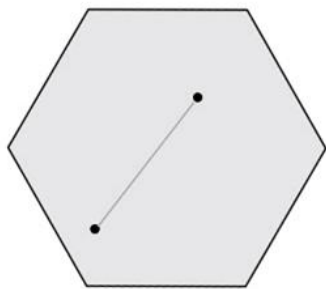
$$x = \theta x_1 + (1 - \theta)x_2$$

$$(\theta \in \mathbf{R})$$

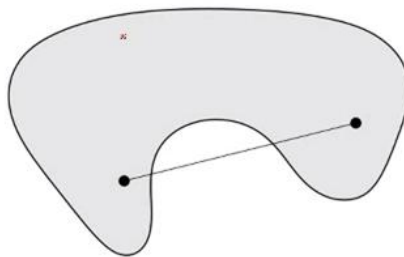
2. 最优化数学基础

• 2.4 凸集

- **定义：** 设集合 $S \subset R^n$ ，如果 $x_1, x_2 \in S$ ，有 $ax_1 + (1 - a)x_2 \in S$ ， $\forall a \in [0,1]$ ，则称 S 为凸集。



凸集



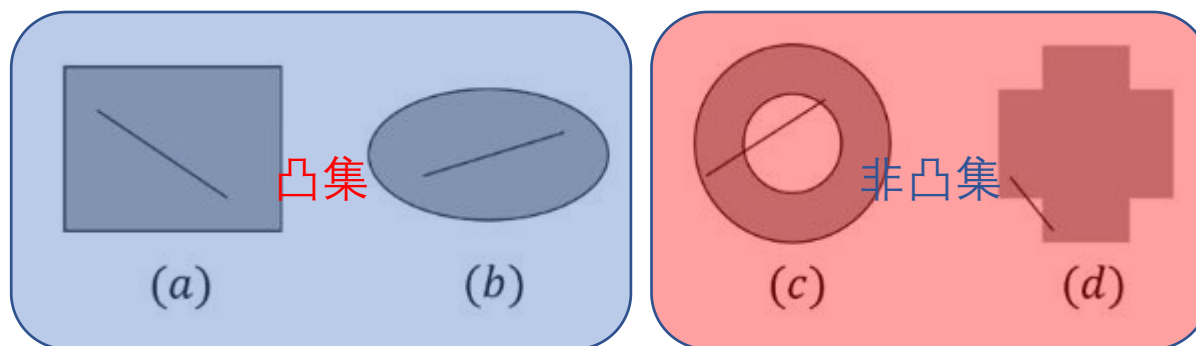
非凸集

- 凸集是指在一个向量空间内，其任意两个元素的线段均在该集合内部的集合。通俗地说，就是集合内不只包含本身元素，还包括这些元素之间所有可能连线的所有点。
- 在优化问题中，许多优化问题都能转化为最小化或最大化凸函数的问题。凸函数在其定义域内的任意一对点之间上凸，这常常是优化问题特殊而有用的性质。利用这一性质，可以设计和实现一些高效且具有实际意义的算法，如凸优化和线性规划。

2. 最优化数学基础

• 2.4 凸集

- **例：**判断以下集合哪些为凸集？



- **注：**连接集合内任意两点的线段仍然完全在该集合内。

2. 最优化数学基础

• 2.4 凸集

• 常见的凸集

单点集： $\{x\}$

空集： $\emptyset$

整个欧式空间： R^n

超平面： $H = \{x \mid P^T x = \alpha\}$

半空间： $H^+ = \{x \mid P^T x \geq \alpha\}$; $H^- = \{x \mid P^T x \leq \alpha\}$

射线： $S = \{x \mid x = x_0 + \lambda d, \lambda \geq 0\}$ d 是非零向量



2. 最优化数学基础

• 2.4 凸集

- **例：**证明射线 $S = \{x | x = x_0 + \lambda d, \lambda \geq 0\}$ 是凸集， d 是非零向量， x_0 为射线顶点。

对任意 $x_1, x_2 \in S$ 和每一个数 $\lambda \in [0, 1]$, 有

$$x_1 = x_0 + \lambda_1 d, \quad x_2 = x_0 + \lambda_2 d,$$

其中 $\lambda_1, \lambda_2 \geq 0$. 因而

$$\begin{aligned} \lambda x_1 + (1 - \lambda)x_2 &= \lambda(x_0 + \lambda_1 d) + (1 - \lambda)(x_0 + \lambda_2 d) \\ &= x_0 + [\lambda\lambda_1 + (1 - \lambda)\lambda_2]d, \end{aligned}$$

$$\lambda\lambda_1 + (1 - \lambda)\lambda_2 \geq 0,$$

$$\boxed{\lambda x_1 + (1 - \lambda)x_2 \in S.} \implies S \text{ 是凸集}$$

注： 梯度下降的一步搜索就是沿射线寻找更优点。



2. 最优化数学基础

• 2.4 凸集

- **例：**证明超球 $\|x\| \leq r$ 为凸集。

证明：设 x, y 为超球中的任意两点， $0 \leq \lambda \leq 1$ ，则有：

$$\begin{aligned} & \|\lambda x + (1 - \lambda)y\| \\ & \leq \lambda\|x\| + (1 - \lambda)\|y\| \\ & \leq \lambda r + (1 - \lambda)r \\ & = r \end{aligned}$$

$$\|\lambda x + (1 - \lambda)y\| \leq r$$

注：正则化约束 $\|w\|_2 \leq r$ 常用于限制模型复杂度，它给参数向量限定一个凸区域，有利于优化和泛化。



2. 最优化数学基础

• 2.5 凸函数 (Convex Function)

- **定义：** 设集合 $S \subset \mathbf{R}^n$ 是非空凸集， $\theta \in [0, 1]$ ， f 是定义在 S 上的函数， 如对任意 $x, y \in S$ ， 有

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

则称 f 是定义在 S 上的凸函数。

- 若：
$$f(\theta x + (1 - \theta)y) < \theta f(x) + (1 - \theta)f(y) \quad \theta \in (0, 1)$$

则称 f 是定义在 S 上的严格凸函数。

- Jensen不等式

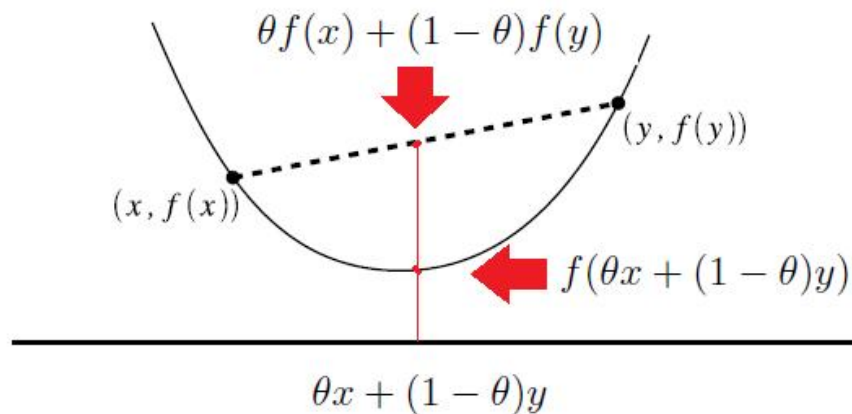
$$f\left(\sum_{i=1}^M \lambda_i x_i\right) \leq \sum_{i=1}^M \lambda_i f(x_i)$$



2. 最优化数学基础

• 2.5 凸函数 (Convex Function)

- **几何意义：**凸函数的图像曲线通常呈现上凸/向上凸起的形状。一元凸函数表示连接函数图形上任意两点之间的连线总是位于曲线弧的上方。



- 凸函数的定义可以看作是将一个点与它自身的两个连线中，连线上的函数值总比点上的函数值或者另一个连线的函数值要小，因此凸函数有很强的单调性和上凸性质。
- 凸函数在最优化问题中起着重要的作用，因为对于许多优化问题，它们都能转化为最小化凸函数的问题。

2. 最优化数学基础

• 2.5 凸函数 (Convex Function)

- **例题：** 设 $f(x) = (x - 1)^2$ ，试证明 $f(x)$ 在 $(-\infty, +\infty)$ 上是严格凸函数。

证明：设 $x, y \in R$ ，且 $x \neq y, 0 < \alpha < 1$ ，则有：

$$\begin{aligned} & f(\alpha x + (1 - \alpha)y) - (\alpha f(x) + (1 - \alpha)f(y)) \\ &= (\alpha x + (1 - \alpha)y - 1)^2 - \alpha(x - 1)^2 - \\ & \quad (1 - \alpha)(y - 1)^2 \\ &= -\alpha(1 - \alpha)(x - y)^2 < 0 \end{aligned}$$

$$\Rightarrow f(\alpha x + (1 - \alpha)y) < (\alpha f(x) + (1 - \alpha)f(y))$$

$$\Rightarrow f(x) \text{ 是严格凸函数}$$

注：一维平方损失是最基本的回归损失；它对应预测值偏离真实值的二次惩罚。



2. 最优化数学基础

• 2.5 凸函数 (Convex Function)

- **例题：**证明线性函数是 R^n 上的凸函数。

$$f(x) = C^T x = c_1 x_1 + c_2 x_2 + \cdots + c_n x_n$$

证明：设 $x, y \in R$ ，且 $x \neq y$, $0 < \alpha < 1$ 则有：

$$\begin{aligned} &= c^T [\alpha x + (1 - \alpha)y] \\ &= \alpha c^T x + (1 - \alpha)c^T y \\ &= \alpha f(x) + (1 - \alpha)f(y) \end{aligned}$$

$$\Rightarrow f(\alpha x + (1 - \alpha)y) = (\alpha f(x) + (1 - \alpha)f(y))$$

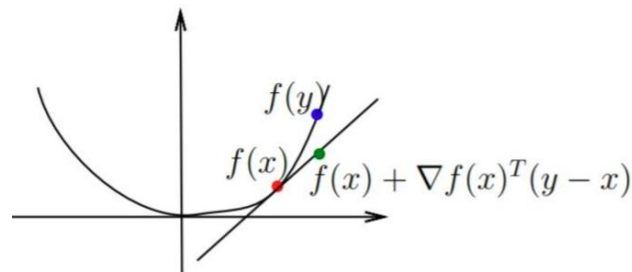
$$\Rightarrow f(x) \text{ 是凸函数}$$

1. 线性函数既是凸函数，也是凹函数
2. 思考并证明： $f(x) = |x|$ 是否为凸函数？
3. $|x|$ 对应 L_1 正则化，能产生稀疏解，是 Lasso、特征选择和模型压缩中的核心工具



2. 最优化数学基础

• 2.5 凸函数 (Convex Function)



• 判定条件:

- **一阶条件:** 设 D 是开凸集, $f(x)$ 在 D 上具有一阶连续导数, 则 $f(x)$ 是 D 上的凸函数的充要条件是: 对 D 上的任意两个不同点 x, y , 恒有:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x)$$

• 证明:

必要性: 设 $x, y \in R^n$, $\alpha \in [0, 1]$, 由 f 的凸性可得

$$f(\alpha y + (1 - \alpha)x) \leq \alpha f(y) + (1 - \alpha)f(x)$$

$$f(x) \approx f(x_0) + \nabla f(x)(x - x_0)$$

泰勒展开

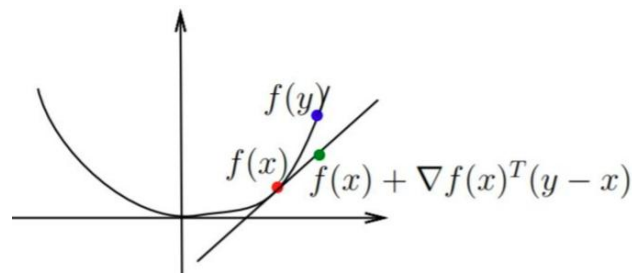
$$f(y) - f(x) \geq \frac{f((x + \alpha(y - x))) - f(x)}{\alpha} \text{ 当 } \alpha \rightarrow 0^+$$

$$\Rightarrow f(y) \geq f(x) + \nabla f(x)^T(y - x)$$

凸函数在任意一点做一阶泰勒展开, 得到的线性函数永远不会超过真实函数值。
即: 切平面是全局下界。

2. 最优化数学基础

• 2.5 凸函数 (Convex Function)



• 判定条件:

- **一阶条件:** 设 D 是开凸集, $f(x)$ 在 D 上具有一阶连续导数, 则 $f(x)$ 是 D 上的凸函数的充要条件是: 对 D 上的任意两个不同点 x, y , 恒有:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x)$$

- **证明:** 充分性: 对任意 $x, y \in R^n$, 下面不等式成立:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x)$$

令 $z = \alpha x + (1 - \alpha)y, \alpha \in [0, 1]$, 则有:

$$f(x) \geq f(z) + \nabla f(z)^T(x - z)$$

$$f(y) \geq f(z) + \nabla f(z)^T(y - z)$$

两式分别乘 α 和 $1 - \alpha$ 再相加:

$$\alpha f(x) + (1 - \alpha)f(y) \geq f(z) = f(\alpha x + (1 - \alpha)y)$$

$\Rightarrow f(x)$ 是凸函数

2. 最优化数学基础

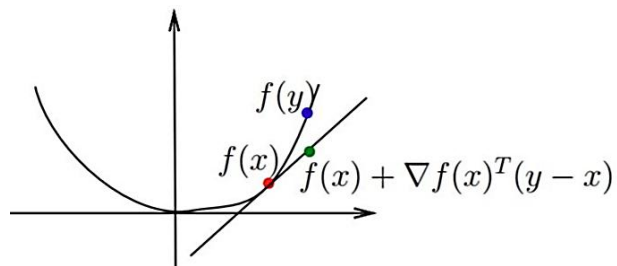
• 2.5 凸函数 (Convex Function)

- 判定条件:

- **一阶条件:** 设 D 是开凸集, $f(x)$ 在 D 上具有一阶连续导数, 则 $f(x)$ 是 D 上的凸函数的充要条件是: 对 D 上的任意两个不同点 x, y , 恒有:

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

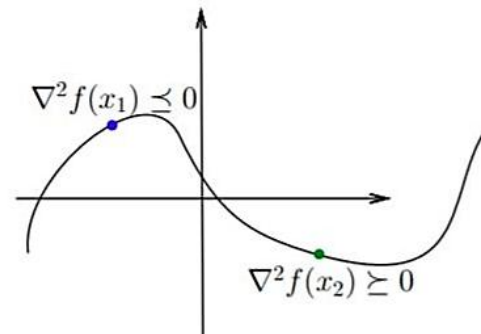
几何表示: 切平面在函数图像下方



- 变形: $f(x)$ 在凸集 D 上可导, 其导函数 $\nabla f(x)$ 在区间凸集 D 上单调递增
- **一阶条件**的一个最重要的推论就是, 如果我们设 $\nabla f(x) = 0$, 那么无论 y 是什么, 都会有 $f(y) \geq f(x)$ 。潜在的意思是说, 对于凸函数做优化, **驻点为 O 就说明找到了极小值**, 因此正如机器学习界经常说的一句话: **当一个问题被证明是一个凸优化问题, 那么基本上就可以算解决了。**
- 很多优化算法用一阶近似建立局部模型, 例如梯度下降用 $\nabla f(x)$ 决定下一步方向。

2. 最优化数学基础

• 2.5 凸函数 (Convex Function)



• 判定条件:

- **二阶条件:** 设 D 是开凸集, $f(x)$ 在 D 上具有二次可微, 则 $f(x)$ 是 D 上的凸函数的充要条件是: $f(x)$ 在 D 上的Hessian矩阵 $D^2 f(x)$ 是半正定的。

- 一个 $n \times n$ 的实对称矩阵 A 是半正定矩阵的充分必要条件是所有的特征值均为非负数。

【定义1】 给定一个大小为 $n \times n$ 的实对称矩阵 A , 若对于任意长度为 n 的非零向量 x , 有 $x^T A x > 0$ 恒成立, 则矩阵 A 是一个正定矩阵。

【定义2】 给定一个大小为 $n \times n$ 的实对称矩阵 A , 若对于任意长度为 n 的向量 x , 有 $x^T A x \geq 0$ 恒成立, 则矩阵 A 是一个半正定矩阵。

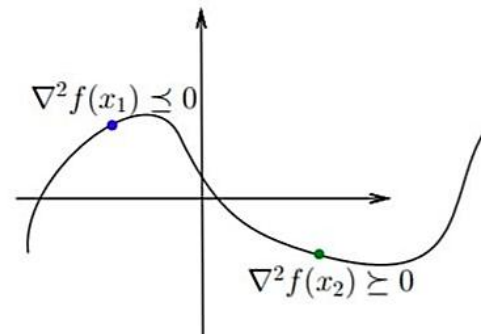
- 利用二阶泰勒展开近似进行证明, 请参考课本P353页定理22.5

$$f(x) = f(a) + f'(a)(x - a) + \frac{1}{2}f''(a)(x - a)^2 + R_2(x)$$



2. 最优化数学基础

• 2.5 凸函数 (Convex Function)



- **判定条件:**

- **二阶条件:** 设 D 是开凸集, $f(x)$ 在 D 上具有二次可微, 则 $f(x)$ 是 D 上的凸函数的充要条件是: $f(x)$ 在 D 上的Hessian矩阵 $D^2f(x)$ 是半正定的。

- 二次函数 $f(x) = \frac{1}{2}x^T Px + q^T x + r$ ($P \in \mathbb{S}^n$)

- 一阶、二阶导数分别为 $\nabla f(x) = Px + q$, $\nabla^2 f(x) = P$
- 当 P 为半正定矩阵时为凸函数

- 最小二乘目标函数: $f(x) = \|Ax - b\|_2^2$

- 一阶、二阶导数分别为 $\nabla f(x) = 2A^T(Ax - b)$, $\nabla^2 f(x) = 2A^T A$
- 对于任意矩阵 A , 最小二乘的目标函数均为凸函数

注: 线性回归就是最小二乘; 神经网络最后一层若固定特征, 训练线性读出层也是一个凸最小二乘问题。

2. 最优化数学基础

复杂凸目标可以由简单凸函数组合得到，例如：

$$\min_w \frac{1}{n} \sum_{i=1}^n \ell_i(w) + \lambda R(w).$$

• 2.5 凸函数 (Convex Function)

- **定理：**如果函数 f ， f_1 和 f_2 都是凸函数，那么对于 $\forall a \geq 0$ ，函数 af 也是凸函数， $f_1 + f_2$ 也是凸函数。

证明：设 $\mathbf{x}, \mathbf{y} \in \Omega$ 和 $\alpha \in (0, 1)$ 。方便起见，记 $\bar{f} = af$ ，由于函数 f 是凸函数，且 $a \geq 0$ ，故有

$$\begin{aligned}\bar{f}(\alpha \mathbf{x} + (1 - \alpha) \mathbf{y}) &= af(\alpha \mathbf{x} + (1 - \alpha) \mathbf{y}) \\ &\leq a(\alpha f(\mathbf{x}) + (1 - \alpha)f(\mathbf{y})) \\ &= \alpha(af(\mathbf{x})) + (1 - \alpha)(af(\mathbf{y})) \\ &= \alpha \bar{f}(\mathbf{x}) + (1 - \alpha)\bar{f}(\mathbf{y})\end{aligned}$$

这意味着函数 $\bar{f}$ 为凸函数。

记 $f_3 = f_1 + f_2$ ，由于 f_1 和 f_2 都是凸函数，故有

$$\begin{aligned}f_3(\alpha \mathbf{x} + (1 - \alpha) \mathbf{y}) &= f_1(\alpha \mathbf{x} + (1 - \alpha) \mathbf{y}) + f_2(\alpha \mathbf{x} + (1 - \alpha) \mathbf{y}) \\ &\leq (\alpha f_1(\mathbf{x}) + (1 - \alpha)f_1(\mathbf{y})) + (\alpha f_2(\mathbf{x}) + (1 - \alpha)f_2(\mathbf{y})) \\ &= \alpha(f_1(\mathbf{x}) + f_2(\mathbf{x})) + (1 - \alpha)(f_1(\mathbf{y}) + f_2(\mathbf{y})) \\ &= \alpha f_3(\mathbf{x}) + (1 - \alpha)f_3(\mathbf{y})\end{aligned}$$

说明函数 f_3 是凸函数。



2. 最优化数学基础

• 2.5 凸函数 (Convex Function)

- **例：**判断 $f(x) = 5x_1^2 + x_2^2 + x_1x_2 - 5x_1 + 4x_2$ 是否为凸函数。

$$\frac{\partial f(x)}{\partial x_1} = 10x_1 + x_2 - 5, \quad \frac{\partial f(x)}{\partial x_2} = x_1 + 2x_2 + 4$$

$$\frac{\partial^2 f(x)}{\partial x_1^2} = 10, \quad \frac{\partial^2 f(x)}{\partial x_1 x_2} = 1$$

$$\frac{\partial^2 f(x)}{\partial x_2^2} = 2, \quad \frac{\partial^2 f(x)}{\partial x_2 x_1} = 1$$

$$\Rightarrow \nabla^2 f(x) = \begin{bmatrix} 10 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\Rightarrow f(x) \text{ 是凸函数}$$

2. 最优化数学基础

• 2.6 凸优化

- **凸优化**：设约束集 $D \in \mathbf{R}^n$ 是凸集，目标函数 f 是定义在 D 上的凸函数，则称此类规划 $\min_{x \in D} f(x)$ 为凸优化问题或凸规划。

$$\begin{array}{ll} \min f(x) & \longrightarrow \text{凸函数} \\ s.t. g_i(x) \leq 0, i = 1, 2, \dots, m & \\ h_j(x) = 0, j = 1, 2, \dots, l & \longrightarrow \text{线性仿射函数} \end{array}$$

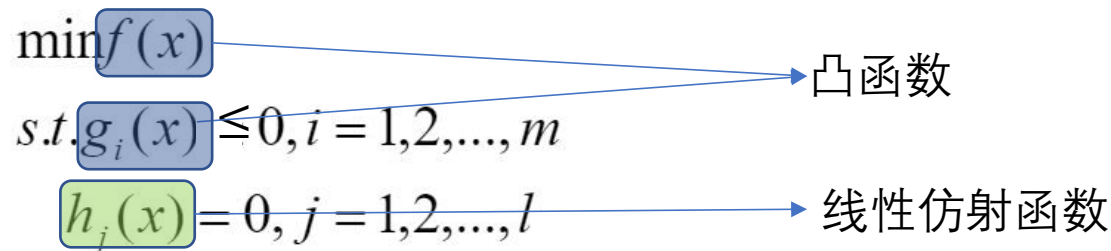
- **目标函数的凸性**：目标函数为凸函数保证了函数的局部最小值就是全局最小值。这是凸优化问题非常重要的性质，因为在非凸优化问题中，可能存在多个局部最小值，找到全局最小值往往是非常困难的。而在凸优化中，我们可以通过各种优化算法较为高效地找到全局最优解。

- 凸优化不等于“目标函数看起来简单”，而是“目标函数凸 + 可行域凸”。
- SVM、逻辑回归、Lasso、最小二乘、很多矩阵补全松弛问题都可以写成凸优化或包含凸优化子问题。

2. 最优化数学基础

• 2.6 凸优化

- **凸优化**：设约束集 $D \in \mathbf{R}^n$ 是凸集，目标函数 f 是定义在 D 上的凸函数，则称此类规划 $\min_{x \in D} f(x)$ 为凸优化问题或凸规划。



- **不等式约束函数的凸性**：凸的不等式约束函数定义了一个凸的可行域。可行域是指所有满足约束条件的 x 的集合。对于凸的不等式约束 $g_i(x) \leq 0$ ，这些约束所确定的区域是凸集，这意味着在可行域内任意两点之间的线段仍然在可行域内。
- **等式约束函数的仿射性**：仿射的等式约束 $h_j(x) = 0$ 也有助于保持可行域的凸性。仿射函数的等值面是一个超平面，超平面是凸集。多个凸集（由不等式约束和等式约束分别确定）的交集仍然是凸集，所以整个可行域是凸集。

2. 最优化数学基础

• 2.6 凸优化

- **凸优化**：设约束集 $D \in \mathbf{R}^n$ 是凸集，目标函数 f 是定义在 D 上的凸函数，则称此类规划 $\min_{x \in D} f(x)$ 为凸优化问题或凸规划。

$$\begin{array}{ll} \min f(x) & \text{凸函数} \\ s.t. g_i(x) \leq 0, i = 1, 2, \dots, m & \\ h_j(x) = 0, j = 1, 2, \dots, l & \text{线性仿射函数} \end{array}$$

- 线性规划、二次规划(目标函数为二次型函数、约束方程为线性方程)都可以归为凸规划。
- 在凸优化问题中，局部极小点就是全局极小点。[证明参考定理22.6]

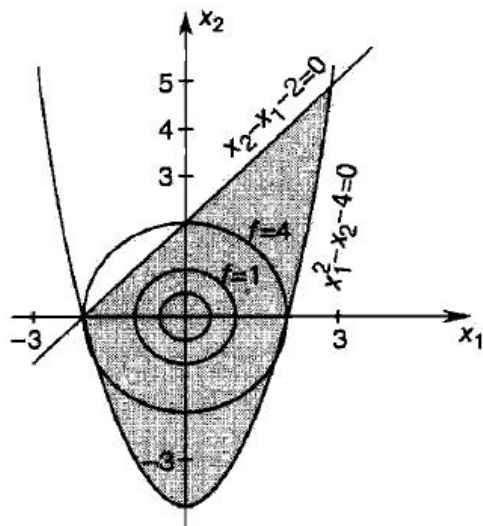


2. 最优化数学基础

• 2.6 凸优化

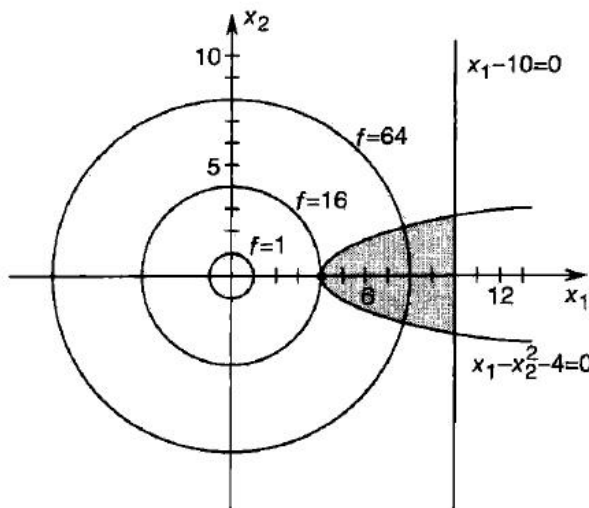
模型训练时加入约束可能改变解的位置。例如加入 $\|w\|_2 \leq r$ 后，最优参数可能被压到球面边界上，对应正则化收缩。

$$\begin{aligned} &\text{minimize} && x_1^2 + x_2^2 \\ &\text{subject to} && x_2 - x_1 - 2 \leq 0 \\ &&& x_1^2 - x_2 - 4 \leq 0 \end{aligned}$$



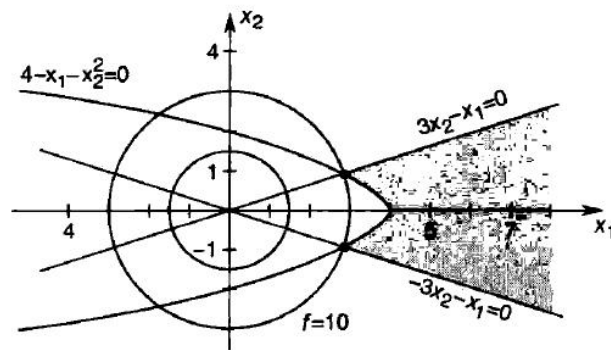
目标函数在有约束条件和无约束条件下极小值相同的情况

$$\begin{aligned} &\text{minimize} && x_1^2 + x_2^2 \\ &\text{subject to} && x_1 - 10 \leq 0 \\ &&& x_1 - x_2^2 - 4 \geq 0 \end{aligned}$$



只有一个约束条件起作用的情况 (等式约束)

$$\begin{aligned} &\text{minimize} && x_1^2 + x_2^2 \\ &\text{subject to} && 4 - x_1 - x_2^2 \leq 0 \\ &&& 3x_2 - x_1 \leq 0 \\ &&& -3x_2 - x_1 \leq 0 \end{aligned}$$



约束条件的存在导致局部极小点的数量增加的情况

2. 最优化数学基础

• 2.6 凸优化

• 为什么凸优化这么重要？

• 凸优化问题的性质：

- 凸优化的**全局优化性质**：**局部极小点就是全局极小点**
- 凸优化目标函数的一阶条件也是充分的，即满足一阶条件的点都是最优解。这一性质称为**一阶最优性条件**。
- 凸优化目标函数的二阶条件也是充分的，即满足二阶条件的点局部是凸的，且是局部最小值点。这一性质称为**二阶最优性条件**。
- 凸优化问题中约束条件也是凸的，则问题的可行域也是凸的，且满足约束条件的点形成的集合也是凸的。这一性质称为**凸可行性条件**。

• 相当一部分问题是或等价于凸优化问题

- 对于非凸优化问题取得的算法理论方面的突破大体其实归结于找到这些非凸优化问题中“凸”的结构（如：深度学习整体非凸，但很多技术仍来自凸优化，包括局部二阶近似、正则化、对偶解释、核方法等）

- 常用的凸优化算法包括梯度下降法、牛顿法、内点法等。



3. 标准形式和对偶形式

• 3.1 对偶问题(Dual Problem)

- **对偶问题**是与原始优化问题（原问题）相对应的一个优化问题，通过对原问题进行对偶化（dualization）得到。
- 对偶问题的定义取决于原问题的形式。原问题可以是线性规划（Linear Programming, LP）、二次规划（Quadratic Programming, QP）等类型。
- 对偶问题的重要性在于它可以通过求解原问题的对偶问题来验证原问题的解是否最优。对于一些问题，对偶问题的形式比原问题更容易求解，而且解也更加直观。
- 最优化中的原问题与对偶问题是相对应的两个优化问题。对于原问题，常规的求解方法是使用各种优化算法来最小化或最大化目标函数，而对于对偶问题，则是使用对应的对偶算法来最小化或最大化对偶函数得到对应的对偶解。



3. 标准形式和对偶形式

• 3.1 对偶问题(Dual Problem)

- 甲方生产计划（生产者视角）

利润最大化:

设备	产品I	产品II	能力
A	2	2	12
B	4	0	16
C	0	5	15
利润	2	3	

$$\begin{aligned} \max \quad & Z = 2x_1 + 3x_2 \\ \left\{ \begin{array}{l} 2x_1 + 2x_2 \leq 12 \\ 4x_1 \leq 16 \\ 5x_2 \leq 15 \\ x_1, x_2 \geq 0 \end{array} \right. \end{aligned}$$

x_1, x_2 是生产两种产品的数量。目标是利润最大化，约束是设备能力不能超限。

3. 标准形式和对偶形式

• 3.1 对偶问题(Dual Problem)

- 资源定价的对偶视角：乙方租用设备

- **甲方：**租金收入应不低于将设备用于自己生产时产生的利润

- 假设A、B、C三种设备的单位租金（单位设备生产单位产品的租金）分别为 y_1 , y_2 , y_3

生产 I 的资源用于出租时，租金收入应满足：

$$2y_1 + 4y_2 \geq 2$$

生产 II 的资源用于出租时，租金收入应满足：

$$2y_1 + 5y_3 \geq 3$$

总租金收入： $12y_1 + 16y_2 + 15y_3$

资源消耗最小化：

$$\min W = 12y_1 + 16y_2 + 15y_3$$

$$s.t \quad 2y_1 + 4y_2 \geq 2$$

$$\Rightarrow \quad 2y_1 + 5y_3 \geq 3$$

$$y_1, y_2, y_3 > 0$$



3. 标准形式和对偶形式

• 3.1 对偶问题(Dual Problem)

某企业计划生产甲、乙两种产品，该两种产品均需经 A、B、C、D 四种不同设备加工，按工艺资料规定，在各种不同设备上的加工时间及设备加工能力、单位产品利润如表中所示。问：如何安排产品的生产计划，才能使企业获利最大？

设备	A	B	C	D	利润
产品甲	2	1	4	0	2
产品乙	2	2	0	4	3
加工能力	12	8	16	12	

3. 标准形式和对偶形式

• 3.1 对偶问题(Dual Problem)

某企业计划生产甲、乙两种产品，该两种产品均需经 A、B、C、D 四种不同设备加工，按工艺资料规定，在各种不同设备上的加工时间及设备加工能力、单位产品利润如表中所示。
问：如何安排产品的生产计划，才能使企业获利最大？

1.最大生产利润模型

设生产甲产品为 x_1 件，乙产品为 x_2 件

$$\begin{aligned} \max \quad & z = 2x_1 + 3x_2 \\ \text{s.t.} \quad & \begin{cases} 2x_1 + 2x_2 \leq 12 \\ x_1 + 2x_2 \leq 8 \\ 4x_1 \leq 16 \\ 4x_2 \leq 12 \\ x_1 \geq 0, x_2 \geq 0 \end{cases} \end{aligned}$$

原问题



对偶问题

2.资源最低售价模型

设第 i 种资源价格为 y_i ($i=1, 2, 3, 4$)

$$\begin{aligned} \min \quad & w = 12y_1 + 8y_2 + 16y_3 + 12y_4 \\ \text{s.t.} \quad & \begin{cases} 2y_1 + y_2 + 4y_3 + 0y_4 \geq 2 \\ 2y_1 + 2y_2 + 0y_3 + 4y_4 \geq 3 \\ y_i \geq 0, (i=1, 2, 3, 4) \end{cases} \end{aligned}$$

原问题每个资源约束对应一个对偶变量；原问题每个产品变量对应一个对偶约束。系数矩阵在对偶中转置。

3. 标准形式和对偶形式

• 3.1 对偶问题(Dual Problem)

	原问题	对偶问题
目标函数	$\max Z = c_1x_1 + c_2x_2 + \dots + c_nx_n$	$\min W = b_1y_1 + b_2y_2 + \dots + b_my_m$
约束条件	$s.t. \begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_m \end{cases}$	$s.t. \begin{cases} a_{11}y_1 + a_{21}y_2 + \dots + a_{m1}y_m \geq c_1 \\ a_{12}y_1 + a_{22}y_2 + \dots + a_{m2}y_m \geq c_2 \\ \vdots \\ a_{1n}y_1 + a_{2n}y_2 + \dots + a_{mn}y_m \geq c_m \end{cases}$
约束变量	$x_j \geq 0; j = 1, 2, \dots, n$	$y_i \geq 0; i = 1, 2, \dots, m$

- 原问题问“给定资源如何赚最多”，对偶问题问“这些资源最低应定价多少，才不会亏”。两个问题在最优时会**对齐**。
- **对偶性**是最优化理论中的重要概念，它将一个问题转化为其“镜像问题”，通过不同视角简化分析或求解。
- 原问题与对偶问题的关系可概括为：**最小化与最大化的对称、变量与约束的对偶、利润与成本的映射**。

3. 标准形式和对偶形式

- 3.2 原问题与对偶问题
 - 不等式约束情况的对偶问题

原问题

$$\max Z = CX$$

$$AX \leq b$$

$$X \geq 0$$

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

$$X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

对偶问题

$$\min W = b^T Y$$

$$A^T Y \geq C^T$$

$$Y \geq 0$$

$$b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

$$C = (c_1, c_2, \cdots, c_n)$$

$$Y = (y_1, y_2, \cdots, y_m)$$

- 符号变换（变量符号与约束条件符号）：“大同小异”



3. 标准形式和对偶形式

- 3.2 原问题与对偶问题

- 不等式约束情况的对偶问题

- 例：写出下列问题的对偶问题

$$\begin{aligned} \min \quad & 3.00x_1 + 2.50x_2 \\ \text{s. t.} \quad & 2x_1 + 4x_2 \geq 40 \\ & 3x_1 + 2x_2 \geq 50 \\ & 2x_1 + x_2 \geq 30 \\ & x_1, x_2 \geq 0 \end{aligned}$$

- 解：

$$\begin{aligned} \max \quad & 40y_1 + 50y_2 + 30y_3 \\ \text{s. t.} \quad & 2y_1 + 3y_2 + 2y_3 \leq 3.00 \\ & 4y_1 + 2y_2 + y_3 \leq 2.50 \\ & y_1, y_2, y_3 \geq 0 \end{aligned}$$



3. 标准形式和对偶形式

- 3.2 原问题与对偶问题
 - 等式/无约束情况的对偶问题

原问题

$$\max \quad Z = CX$$

$$s.t. \quad \begin{cases} AX = b \\ X \geq 0 \end{cases} \Rightarrow$$

对偶问题

$$\min \quad W = Yb$$

$$s.t. \quad \begin{cases} YA \geq C \\ Y \text{ 无约束} \end{cases}$$



3. 标准形式和对偶形式

• 3.2 原问题与对偶问题

- 等式/无约束情况的对偶问题

- 例：写出下列问题的对偶问题

$$\begin{aligned} \min \quad & 2x_1 - 3x_2 + 4x_3 \\ \text{s. t.} \quad & 2x_1 + 3x_2 - 5x_3 = 2 \\ & 3x_1 + x_2 + 7x_3 = 3 \\ & -x_1 + 4x_2 + 6x_3 = 5 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

- 解：

$$\begin{aligned} \max \quad & 2y_1 + 3y_2 + 5y_3 \\ \text{s. t.} \quad & 2y_1 + 3y_2 - y_3 \leq 2 \\ & 3y_1 + y_2 + 4y_3 \leq -3 \\ & -5y_1 + 7y_2 + 6y_3 \leq 4 \\ & y_1, y_2, y_3 \text{ 无约束} \end{aligned}$$

3. 标准形式和对偶形式

- 3.2 原问题与对偶问题
 - 一般线性规划的对偶问题

原问题

$$\begin{aligned} \max \quad & c_1 x_1 + c_2 x_2 \\ \text{s.t.} \quad & a_{11} x_1 + a_{12} x_2 \leq b_1 \\ & a_{21} x_1 + a_{22} x_2 = b_2 \\ & a_{31} x_1 + a_{32} x_2 \geq b_3 \\ & x_1 \geq 0, x_2 \text{ 无约束} \end{aligned}$$



对偶问题

$$\begin{aligned} \min \quad & b_1 y_1 + b_2 y_2 + b_3 y_3 \\ \text{s.t.} \quad & a_{11} y_1 + a_{21} y_2 + a_{31} y_3 \geq c_1 \\ & a_{12} y_1 + a_{22} y_2 + a_{32} y_3 = c_2 \\ & y_1 \geq 0, y_3 \leq 0, y_2 \text{ 无约束} \end{aligned}$$



3. 标准形式和对偶形式

• 3.2 原问题与对偶问题

原问题（或对偶问题）	对偶问题（或原问题）
目标函数 MaxZ	目标函数 MinW
约束条件数：m个	对偶变量数：m个
第i个约束条件类型为“ $\leq$ ”	第i个变量 ≥ 0
第i个约束条件类型为“ $\geq$ ”	第i个变量 ≤ 0
第i个约束条件类型为“ $=$ ”	第i个变量是自由变量
决策变量数：n个	约束条件数：n
第j个变量 ≥ 0	第j个约束条件类型为“ $\geq$ ”
第j个变量 ≤ 0	第j个约束条件类型为“ $\leq$ ”
第j个变量是自由变量	第j个约束条件类型为“ $=$ ”

- 符号变换（变量符号与约束条件符号）：“大同小异”

3. 标准形式和对偶形式

• 3.2 原问题与对偶问题

求极大值 左边求极大，右边求极小 求极小值

$$\begin{array}{ll} \max & Z = c_1x_1 + c_2x_2 + c_3x_3 \\ \text{s.t.} & \begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 \leq b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 \geq b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \\ x_1 \geq 0, x_2 \leq 0, x_3 \text{无限制} \end{cases} \end{array}$$
$$\begin{array}{ll} \min & W = b_1y_1 + b_2y_2 + b_3y_3 \\ \text{s.t.} & \begin{cases} a_{11}y_1 + a_{21}y_2 + a_{31}y_3 \geq c_1 \\ a_{12}y_1 + a_{22}y_2 + a_{32}y_3 \leq c_2 \\ a_{13}y_1 + a_{23}y_2 + a_{33}y_3 = c_3 \\ y_1 \geq 0, y_2 \leq 0, y_3 \text{无限制} \end{cases} \end{array}$$

无限制变等号，等号变无限制，其他符号相反。

无限制变等号，等号变无限制，其他符号相同。

- 1、原问题有几个约束，对偶问题就有几个变量
- 2、原来的约束系数矩阵要转个置
- 3、原来的目标函数系数变约束常量，原来的约束常量变目标系数
- 4、按照这个模板带进去就可以了

• 符号变换（变量符号与约束条件符号）：“大同小异”

3. 标准形式和对偶形式

• 3.2 原问题与对偶问题

• 一般线性规划的对偶问题

• 例：写出下列问题的对偶问题

$$\begin{aligned} \min Z &= 2x_1 + 3x_2 - 5x_3 + x_4 \\ \begin{cases} x_1 + x_2 - 3x_3 + x_4 \geq 5 \\ 2x_1 + 2x_3 - 4x_4 \leq 4 \\ x_2 + x_3 + x_4 = 6 \\ x_1 \leq 0, x_2, x_3 \geq 0, x_4 \text{无约束} \end{cases} \end{aligned}$$

• 解：

$$\begin{aligned} \max W &= 5y_1 + 4y_2 + 6y_3 \\ \begin{cases} y_1 + 2y_2 \geq 2 \\ y_1 + y_3 \leq 3 \\ -3y_1 + 2y_2 + y_3 \leq -5 \\ y_1 - 4y_2 + y_3 = 1 \\ y_1 \geq 0, y_2 \leq 0, y_3 \text{无约束} \end{cases} \end{aligned}$$



3. 标准形式和对偶形式

• 3.2 原问题与对偶问题

• 一般线性规划的对偶问题

• 例：写出下列问题的对偶问题

$$\begin{aligned} \min Z &= 3x_1 + 2x_2 - 3x_3 + 4x_4 \\ \begin{cases} x_1 - 2x_2 + 3x_3 + 4x_4 \leq 3 \\ x_2 + 3x_3 + 4x_4 \geq -5 \\ 2x_1 - 3x_2 - 7x_3 - 4x_4 = 2 \\ x_1 \geq 0, \quad x_2 \leq 0 \end{cases} \end{aligned}$$

• 解：

$$\begin{aligned} \max W &= 3y_1 - 5y_2 + 2y_3 \\ \begin{cases} y_1 + 2y_3 \leq 3 \\ -2y_1 + y_2 - 3y_3 \geq 2 \\ 3y_1 + 3y_2 - 7y_3 = -3 \\ 4y_1 + 4y_2 - 4y_3 = 4 \\ y_1 \leq 0, \quad y_2 \geq 0 \end{cases} \end{aligned}$$



3. 标准形式和对偶形式

• 3.2 原问题与对偶问题

• 一般线性规划的对偶问题---应用

- **资源分配问题：**在一个公司中，有一些资源（如人力、设备等）需要被分配到各个项目中，以实现**最大的利润**。线性规划常用于优化资源分配问题。对于其中的对偶问题，它可以用于**最小化资源使用成本**，如最小化设备使用成本、最小化人力成本等。
- **生产计划问题：**在一个制造企业中，需要通过合理的生产计划来**最大化产量和利润**。线性规划可用于计算生产不同产品的最优数量，以满足不同市场需求。对于每个产品，对偶问题可用于**最小化生产成本**。
- **运输问题：**在一个物流公司中，需要决定如何通过合适的运输方式（如货车、火车、飞机等）将货物从一个地方运往另一个地方。线性规划可用于优化运输计划，以确保货物以**最小的成本和时间**运送到目的地。对于运输成本的对偶问题可以应用于**最大化货运收益**。
- **市场竞争问题：**在一个市场上，多种企业竞争同一个市场份额。线性规划可用于确定每个企业应该如何设置产品价格和生产量以**最大化其利润**。对偶问题可用于**最小化成本**，如材料成本、劳动力成本等。
- **能源生产问题：**在能源生产领域中，需要通过调整能源生产数量来满足市场需求。线性规划可应用于优化生产计划，同时**满足能源需求和盈利目标**。对偶问题可以应用于**最小化生产成本**，如最小化能源生产成本。



3. 标准形式和对偶形式

• 3.2 原问题与对偶问题

- 原问题和对偶问题的概念并不仅限于线性规划问题（LP），它们同样适用于其他类型的优化问题。
- 其他可以定义对偶问题的优化问题：
 - 二次规划问题（Quadratic Programming, QP）：原问题可能涉及到一个二次的目标函数和线性约束。对偶问题同样会有二次形式，但目标函数和约束之间的关系与线性规划不同。
 - 整数规划问题（Integer Programming, IP）：原问题可能包含整数或二元决策变量。对偶问题虽然可以定义，但整数性质使得对偶问题可能不如线性规划中的对偶问题那样直接有用；
 - 混合整数规划问题（Mixed Integer Programming, MIP）：结合了线性规划和整数规划的特点。对偶问题可以定义，但在求解时需要特别注意整数约束的处理。
 - 动态规划问题：原问题通常涉及多阶段决策过程。对偶问题可能涉及不同阶段的资源分配或成本最小化；
 - 半定规划问题（Semidefinite Programming, SDP）：原问题涉及到半正定矩阵的优化。对偶问题同样涉及到半正定矩阵，但目标函数和约束的性质有所不同；
 - 几何规划问题（Geometric Programming, GP）：原问题涉及单调递增或递减的函数。对偶问题可以定义，并且有助于解决某些特定类型的优化问题。
- LP问题中的对偶性是一个特别重要和广泛研究的领域，因为它具有一些独特的性质和成熟的对偶理论，如**强对偶性**和对偶问题的可行性与原问题可行性之间的关系。

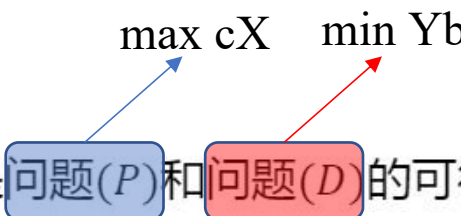


3. 标准形式和对偶形式

• 3.3 对偶问题的性质

• 弱对偶定理

设 $\bar{x}$ 和 $\bar{y}$ 分别是问题(P)和问题(D)的可行解 $\bar{X}, \bar{Y}$, 则必有:



$$C\bar{X} \leq \bar{Y}b; \quad \text{即} \quad \sum_{j=1}^n c_j x_j \leq \sum_{i=1}^m y_i b_i$$

- 弱对偶性质通常总是成立的, 无论原问题和对偶问题是否可行。
- 任何合理定价都不会低于可获得利润

• 强对偶定理

若一对对偶问题 (P) 和 (D) 都有可行解, 则它们都有最优解, 且目标函数的最优值必相等。

$$C\bar{X} = \bar{Y}b;$$

- 强对偶性质并不总是成立, 它依赖于特定类型的优化问题和问题的条件。
- 最优生产利润等于最低合理资源定价

3. 标准形式和对偶形式

• 3.3 对偶问题的性质

- 弱对偶定理

- 强对偶定理

- **关系：**弱对偶性质是强对偶性质的一个必要条件，但不是充分条件。也就是说，强对偶性质成立时，弱对偶性质一定成立；但弱对偶性质成立时，强对偶性质不一定成立。强对偶性质提供了原问题和对偶问题之间的更深层次的联系，它允许我们通过求解一个较容易的问题来找到另一个问题的最优解。
- 在线性规划中，如果原问题和对偶问题都是可行的，且至少有一个问题是有界的（即目标函数有最大值或最小值），那么强对偶性质成立。



3. 标准形式和对偶形式

• 3.3 对偶问题的性质

• 支持向量机 (SVM) 与 (拉格朗日) 对偶问题

- SVM 原问题是在分类间隔最大化和误分类惩罚之间做优化，通过拉格朗日乘子把约束放进目标函数，得到对偶问题。

$$\begin{aligned} & \min_{w,b} \frac{1}{2} \|\mathbf{w}\|^2 \\ & s.t. \quad y_i(\mathbf{w}_i^T \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, 3, \dots, m \end{aligned}$$

↓

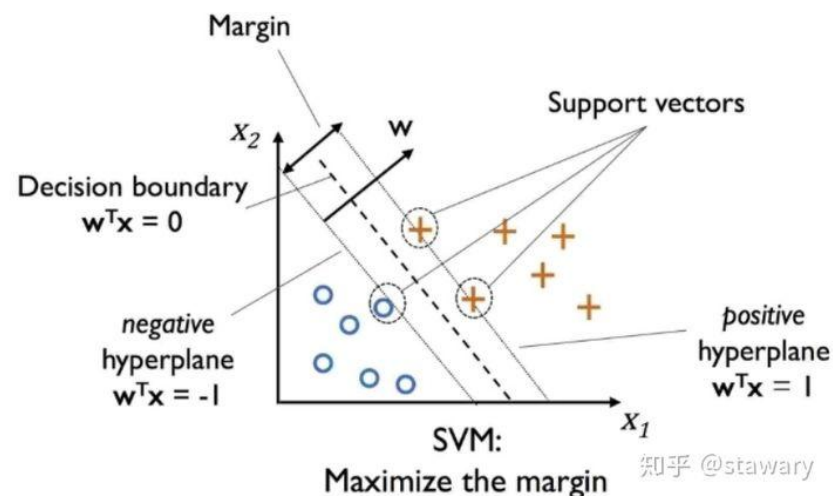
$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

↓

$$\min_{w,b} \max_{\alpha} L(w, b, \alpha)$$

↓

$$\max_{\alpha} \min_{w,b} L(w, b, \alpha)$$



参考资料: [支持向量机\(SVM\)-- 拉格朗日对偶\(Lagrange duality\)](#)



4. 连续优化

- 最小二乘法
- 一阶优化算法
 - 梯度下降法
 - 最速梯度下降法
 - 随机梯度下降法
 - 共轭梯度法
- 二阶优化算法
 - 牛顿法
 - 拟牛顿法



4. 连续优化

- 4.1 最小二乘法
 - 最小二乘法的基本原理
 - 线性回归方程解法



4. 连续优化

• 4.1 最小二乘法

- **定义：**如果有n个点 $(x_1, y_1), (x_2, y_2) \dots (x_n, y_n)$, 可以用如下表达式来刻画这些点与直线 $y = wx + b$ 的接近程度：

$$[y_1 - (wx_1 + b)]^2 + [y_2 - (wx_2 + b)]^2 + \dots + [y_n - (wx_n + b)]^2$$

使得上面表达式达到**最小值**的直线 $y = f(x) = wx + b$ 即为我们要求的直线，此种方法称为**最小二乘法**。

$$\min \sum_{i=1}^n (y_i - f(x_i))^2$$

$$(w^*, b^*) = \operatorname{argmin}_{(w, b)} \sum_{i=1}^n (y_i - f(x_i))^2$$

$$(w^*, b^*) = \operatorname{argmin}_{(w, b)} \sum_{i=1}^n (y_i - wx_i - b)^2$$



4. 连续优化

• 4.1 最小二乘法

- **应用：**利用最小二乘法估计时，要先做出数据的散点图，若散点图呈线性关系，可以用最小二乘法估计出线性回归方程；若散点图呈现出其他曲线关系，则需要利用其他工具进行拟合。

4. 连续优化

• 4.1 最小二乘法

• 线性回归方程

- $y = f(x) = a + bx$
- 目标函数: $\min L = \sum_{i=1}^n (y_i - f(x_i))^2 = \sum_{i=1}^n (y_i - (a + bx_i))^2$

如果用 $\bar{x}$ 表示 $\frac{x_1 + x_2 + \dots + x_n}{n}$, 用 $\bar{y}$ 表示 $\frac{y_1 + y_2 + \dots + y_n}{n}$,

则可以求得 $b =$

$$\frac{(x_1 - \bar{x})(y_1 - \bar{y}) + (x_2 - \bar{x})(y_2 - \bar{y}) + \dots + (x_n - \bar{x})(y_n - \bar{y})}{(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2}$$
$$= \frac{x_1 y_1 + x_2 y_2 + \dots + x_n y_n - n \bar{x} \bar{y}}{x_1^2 + x_2^2 + \dots + x_n^2 - n \bar{x}^2}. \quad a = \underline{\bar{y} - b \bar{x}}$$

- [具体推导过程]
- 直线方程 $y = a + bx$ 为 **线性回归方程**, a, b 为该线性回归方程的系数。

- 最小二乘估计的优点在于它是可解释的、计算简单、可扩展, 并且可以推广到多元回归问题。



4. 连续优化

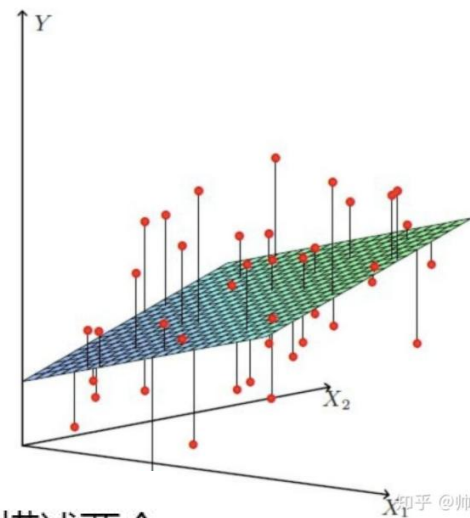
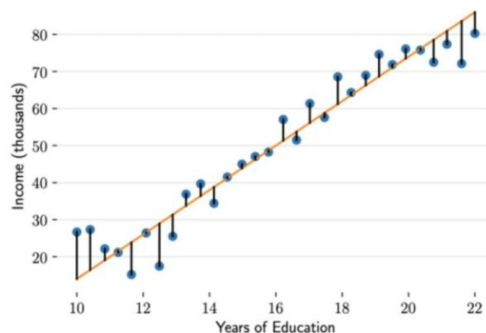
• 4.1 最小二乘法

• 线性回归方程的应用：

(1)描述两变量之间的依存关系；利用线性回归方程即可定量描述两个变量间依存的数量关系。

(2)利用回归方程进行预测或规定 y 值的变化，通过控制 x 的范围来实现目标。如已经得到了空气中NO的浓度和汽车流量间的回归方程，即可通过控制汽车流量来控制空气中的NO的浓度。

(3)注意作回归分析要有实际意义，回归分析前，最好先作出散点图，确定合适的拟合模型。



4. 连续优化

• 4.1 最小二乘法

• 线性回归方程的求解步骤：

- **Step 1:** 列表 $x_i, y_i, x_i y_i$;
- **Step 2:** 计算 $\bar{x}, \bar{y}, \sum_{i=1}^n x_i^2, \sum_{i=1}^n x_i y_i$;
- **Step 3:** 代入公式, 计算 b 和 a ;

$$b = \frac{x_1 y_1 + x_2 y_2 + \cdots + x_n y_n - n \bar{x} \bar{y}}{x_1^2 + x_2^2 + \cdots + x_n^2 - n \bar{x}^2}$$

$$a = \bar{y} - b \bar{x}$$

- **Step 4:** 写出线性回归方程 $y = a + bx$;

• 利用回归直线对总体进行估计

- 利用回归直线进行预测：若回归方程为 $y = a + bx$, 则在 $x = x_0$ 处的估计值为 $y = a + bx_0$

4. 连续优化

• 4.1 最小二乘法

【例题】 某种产品的广告费 x (单位: 百万元)与销售额 y (单位: 百万元)之间有如下对应数据:

x	2	4	5	6	8
y	30	40	60	50	70

(1)画出散点图;

(2)判断 x 与 y 是否具有线性相关关系, 若具有, 求回归直线方程, 并说明回归直线方程斜率的意义.

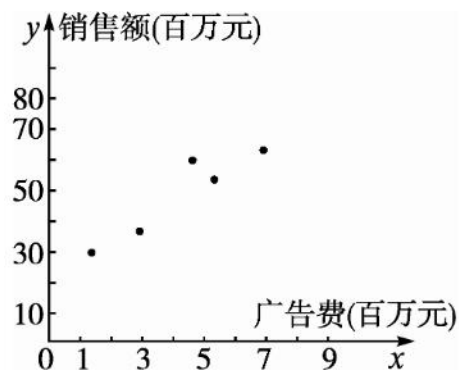
4. 连续优化

• 4.1 最小二乘法

【例题】 某种产品的广告费 x (单位: 百万元)与销售额 y (单位: 百万元)之间有如下对应数据:

x	2	4	5	6	8
y	30	40	60	50	70

解 (1)散点图如图所示



(2)由散点图知 x 与 y 具有线性相关关系. 由题中数据可求得 $\bar{x}=5$, $\bar{y}=50$,

$$\sum_{i=1}^5 x_i y_i = 1\,380, \quad \sum_{i=1}^5 x_i^2 = 145,$$

$$\text{所以 } b = \frac{\sum_{i=1}^5 x_i y_i - 5\bar{x} \cdot \bar{y}}{\sum_{i=1}^5 x_i^2 - 5\bar{x}^2} = \frac{1\,380 - 5 \times 5 \times 50}{145 - 5 \times 5^2} = 6.5,$$

$$a = \bar{y} - b\bar{x} = 50 - 6.5 \times 5 = 17.5.$$

所求回归直线方程为 $y = 6.5x + 17.5$.

斜率的意义?



4. 连续优化

• 4.1 最小二乘法

最小二乘法也可以用于曲线拟合问题。曲线拟合是指给定一组离散的数据点，通过一条曲线去近似这些数据点。在实际应用中，曲线拟合问题经常出现在信号处理、图像处理、机器学习等领域。

• 曲线拟合问题：

- 在处理数据时，一般需把实验数据描成曲线以反映物理量之间的关系
- 为了使曲线能代替数据点的分布规律，则要求所描曲线是平滑的，既要尽可能使各数据点对称且均匀分布在曲线两侧。
- 由于目测有误差，所以同一组数据点不同的实验者可能描成几条不同曲线（或直线），且似乎都能满足上述平滑条件。那么，究竟哪一条曲线是最佳拟合曲线呢？这就是**曲线拟合问题**。
 - **任务1**：物理量 y 与观测量 x 之间的函数关系已确定，只有其中的常数未定，需根据数据点拟合出各常数的最佳值。
 - **任务2**：物理量 y 与观测量 x 之间的函数关系未知，需从函数点拟合出 y 与 x 函数关系的经验公式，以及求出各个常数的最佳值。



4. 连续优化

• 4.1 最小二乘法

- 最小二乘法通过求导找极值点，直接给出参数的解析解，但存在以下限制：
 - **模型线性约束**：仅适用于线性模型或简单的曲线拟合问题，无法处理复杂的非线性模型
 - **凸函数前提**：损失函数必须是凸函数（如均方误差），若遇到非凸函数（如神经网络的交叉熵本身对逻辑回归参数可形成凸问题，但与深层神经网络组合后通常非凸），解析解失效；
 - **计算复杂度瓶颈**：当特征数 m 极大时（如百万级特征），求解矩阵逆 $(X^T X)^{-1}$ 的计算量呈指数级增长。

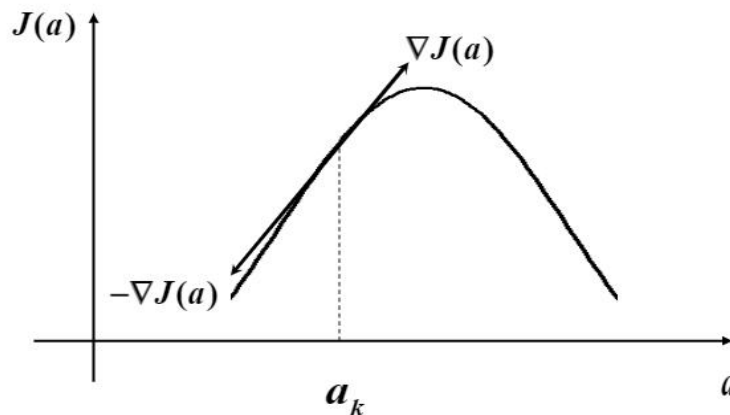
用“迭代寻优”替代“直接求解”



4. 连续优化

• 4.2 梯度下降法

- **梯度下降法**又称**最速下降法**，函数 $J(a)$ 在某点 a_k 的梯度 $\nabla J(a_k)$ 是一个向量，其方向为 $J(a)$ 在该点增长最快的方向。显然负梯度方向 $-\nabla J(a_k)$ 为 $J(a)$ 减少最快的方向。
- 在梯度下降法中，求函数极大值时，沿着梯度方向走，可以最快到达极大点；反之，沿着负梯度方向走，则最快到达极小点。



4. 连续优化

• 4.2 梯度下降法

• 搜索方向

- 求函数 $J(a)$ 极小值的问题，可以选择任意初始点 a_0 ，从 a_0 出发沿着负梯度方向走，可使得 $J(a)$ 下降最快。

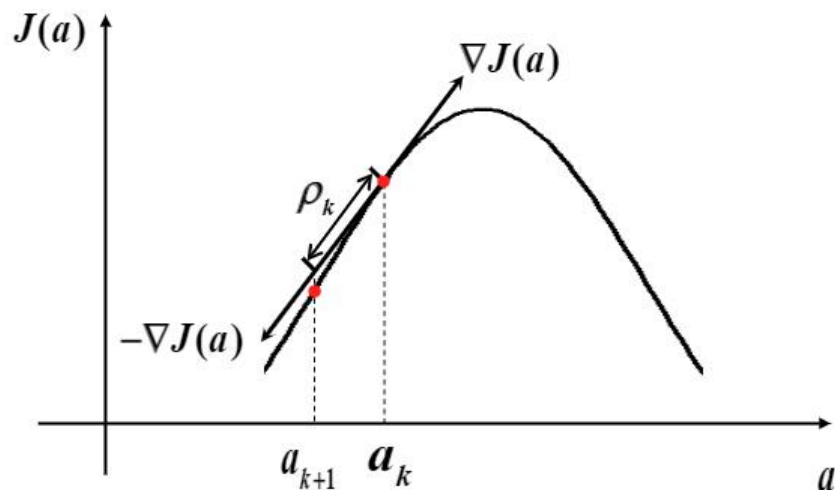
$$s^{(0)} = -\nabla J(a_0)$$

- $s^{(0)}$: 点 a_0 的搜索方向。

4. 连续优化

• 4.2 梯度下降法

• 搜索方向



- 对于任意点 a_k ，定义 a_k 点的负梯度搜索方向的单位向量为：

$$\hat{\mathbf{s}}^{(k)} = -\frac{\nabla J(a_k)}{\|\nabla J(a_k)\|}$$

- 从 a_k 点出发，沿着 $\hat{\mathbf{s}}^{(k)}$ 方向走一步，步长为 ρ_k ，得到的新点 a_{k+1} 表示为：

$$a_{k+1} = a_k + \rho_k \hat{\mathbf{s}}^{(k)}$$

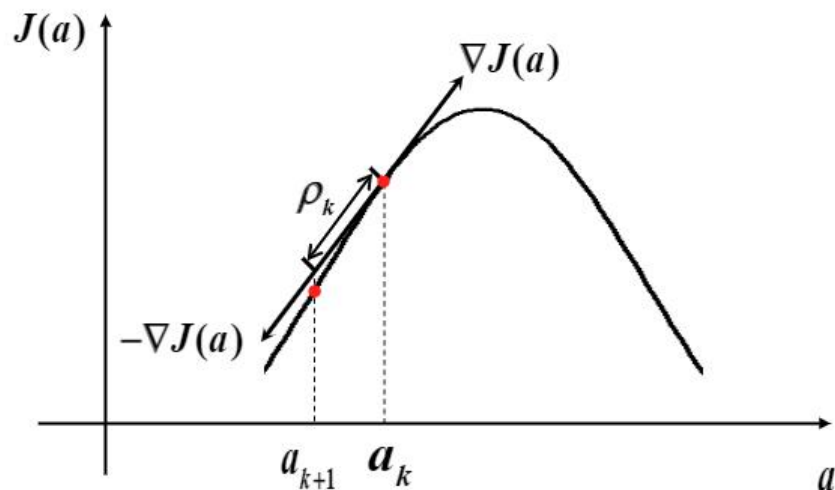
因此，在新点 a_{k+1} ，函数 $J(a)$ 的函数值为：

$$J(a_{k+1}) = J(a_k + \rho_k \hat{\mathbf{s}}^{(k)})$$

4. 连续优化

• 4.2 梯度下降法

• 搜索方向



- 所有的 a_k 组成一个序列，该序列由迭代算法生成：

$$a_0, a_1, a_2, \dots, a_k, a_{k+1}, \dots$$

- 该序列在一定条件下收敛域使得 $J(a)$ 最小的解 a^*
- 迭代算法公式：

$$a_{k+1} = a_k + \rho_k \hat{S}^{(k)}$$

4. 连续优化

• 4.2 梯度下降法

• 最速梯度算法步骤:

- ① 给定初始点 $x^1 \in R^n$, 允许误差 $\varepsilon > 0$, 令 $k = 1$;
- ② 计算搜索方向 $d^k = -\nabla f(x^k)$;
- ③ 若 $\|d^k\| \leq \varepsilon$, 则停止计算, x^k 为所求极值点, 否则, 求最优步长 λ_k , 使得 $f(x^k + \lambda_k d^k) = \min_{\lambda} f(x^k + \lambda d^k)$;
- ④ 令 $x^{k+1} = x^k + \lambda_k d^k$, 令 $k = k + 1$, 转向步骤2.

4. 连续优化

• 4.2 梯度下降法

• 梯度法（最速梯度下降法）

- 搜索方向 $d^k = -\nabla f(x^k)$ ，也称为最速下降方向；

- 搜索步长： λ_k 取最优步长，即满足
$$f(x^k + \lambda_k d^k) = \min_{\lambda} f(x^k + \lambda d^k);$$

• 关键问题：如何设计步长？

- 如果步长太小，则算法收敛慢；

- 如果步长太大，则可能会导致发散。



4. 连续优化

• 4.2 梯度下降法

• 一维搜索

- 采用数学规划法求函数极值点的迭代计算：

$$x^{k+1} = x^k + \lambda_k d^k$$

d^k 次迭代的搜索方向

搜索的最佳步长因子

- 当搜索方向 d^k 给定，求最佳步长 λ_k 就是求一元函数的极小值

$$f(x^{k+1}) = f(x^k + \lambda_k d^k) = \varphi(\lambda_k)$$

- 此方法称为一维搜索，是优化搜索方法的基础。
- 求解一元方程 $\varphi(\lambda)$ 的极小点 λ^* 可用解析法。

4. 连续优化

• 4.2 梯度下降法

• 一维搜索

• 解析法：

$$\begin{aligned} f(x + \lambda d) &\approx f(x) + \lambda d^T \nabla f(x) + \frac{1}{2} (\lambda d)^T G (\lambda d) \\ &= f(x) + \lambda d^T \nabla f(x) + \frac{1}{2} \lambda^2 d^T G d \end{aligned}$$

2阶泰勒展开

• 上式求 λ 的极值，即求 λ 导数为零。

$$d^T \nabla f(x) + \lambda d^T G d = 0$$

则：

$$\lambda^* = - \frac{d^T \nabla f(x)}{d^T G d}$$

• 数值法：确定 λ^* 的搜索区间，再不断缩小区间，最终获得近似值。 如线搜索（精确线搜索、Armijo线搜索以及Wolfe线搜索）

4. 连续优化

• 4.2 梯度下降法

- 极小点的一阶必要条件：一阶梯度为0
- 极小点的二阶必要条件：一阶梯度为0，二阶黑塞矩阵正定

- **例：**用最速下降法求解

$$\min f(x_1, x_2) = x_1^3 + x_2^3 - 3x_1x_2 \quad \mathbf{r}_{x_0} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

迭代2次判别所得的点是否为极小点？

- **解：**

$$\nabla f(\mathbf{r}_{x_1}) = \begin{bmatrix} 3x_1^2 - 3x_2 \\ -3x_1 + 3x_2^2 \end{bmatrix} \quad G(x_1, x_2) = \begin{bmatrix} 6x_1 & -3 \\ -3 & 6x_2 \end{bmatrix}$$

$$\mathbf{r}_{g_0} = \begin{bmatrix} 3 \\ -3 \end{bmatrix} \quad \mathbf{r}_{p_0} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \quad \mathbf{r}_x = \begin{bmatrix} 1 \\ 0 \end{bmatrix} + t \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1-t \\ t \end{bmatrix}$$

$$\varphi(t) = (1-t)^3 + t^3 - 3(1-t)t = 6t^2 - 6t + 1$$

负梯度方向



4. 连续优化

• 4.2 梯度下降法

- 极小点的一阶必要条件：一阶梯度为0
- 极小点的二阶必要条件：一阶梯度为0，二阶黑塞矩阵正定

- **例：**用最速下降法求解

$$\min f(x_1, x_2) = x_1^3 + x_2^3 - 3x_1x_2 \quad \mathbf{r}_{x_0} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

迭代2次判别所得的点是否为极小点？

- **解：**

$$\varphi'(t) = 12t - 6 = 0, t = \frac{1}{2}, \mathbf{r}_{x_1} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}.$$

$$\mathbf{r}_{g_1} = \begin{bmatrix} -3/4 \\ -3/4 \end{bmatrix} \neq 0, \quad \mathbf{r}_{x_1} \text{ 不是极小点, 继续迭代}$$

在第一次迭代后一阶导数不为0

$$\text{取 } \mathbf{r}_{p_1} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{r}_x = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix} + t \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/2 + t \\ 1/2 + t \end{bmatrix}$$

$$\varphi(t) = 2(1/2 + t)^3 - 3(1/2 + t)^2$$

$$\varphi'(t) = 6(1/2 + t)^2 - 6(1/2 + t) = 0, t = \frac{1}{2}, t = -\frac{1}{2} \text{ (舍去)} \quad \text{步长 } t > 0$$



4. 连续优化

• 4.2 梯度下降法

- 极小点的一阶必要条件：一阶梯度为0
- 极小点的二阶必要条件：一阶梯度为0，二阶黑塞矩阵正定

- **例：**用最速下降法求解

$$\min f(x_1, x_2) = x_1^3 + x_2^3 - 3x_1x_2 \quad \mathbf{r}_{x_0} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

迭代2次判别所得的点是否为极小点？

- **解：**

$$\mathbf{r}_{x_2} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{g}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, G(1,1) = \begin{bmatrix} 6 & -3 \\ -3 & 6 \end{bmatrix}.$$

$\mathbf{r}_{x_2}$ 是驻点，G正定， $\mathbf{r}_{x_2}$ 是严格局部极小。

4. 连续优化

• 4.2 梯度下降法

- **例：**用最速下降法求解： $\min f(x) = x_1^2 + 3x_2^2$ ，设初始点为 $x^1 = (2, 1)^T$ ，求迭代一次后的迭代点 x^2 。

• **解：**

$$\ominus \quad \nabla f(x) = (2x_1, 6x_2)^T,$$

$$\therefore d^1 = -\nabla f(x^1) = (-4, -6)^T.$$

$$\therefore x^1 + \lambda d^1 = (2 - 4\lambda, 1 - 6\lambda)^T.$$

$$\text{令 } \varphi(\lambda) = f(x^1 + \lambda d^1) = (2 - 4\lambda)^2 + 3(1 - 6\lambda)^2,$$

$$\text{求解 } \min_{\lambda} \varphi(\lambda)$$

$$\text{令 } \varphi'(\lambda) = -8(2 - 4\lambda) - 36(1 - 6\lambda) = 0 \Rightarrow \lambda_1 = \frac{13}{62}$$

$$\therefore x^2 = x^1 + \lambda_1 d^1 = \left(\frac{36}{31}, \frac{-8}{31}\right)^T$$



4. 连续优化

• 4.2 梯度下降法

• 收敛性

- 性质： 设 $f(x)$ 有一阶连续偏导数，若步长 λ_k 满足

$$f(x^k + \lambda_k d^k) = \min_{\lambda} f(x^k + \lambda d^k)$$

则有 $\nabla f(x^k + \lambda_k d^k)^T d^k = 0$ 。

- 证明： 令 $\varphi(\lambda) = f(x^k + \lambda d^k)$, 所以

$$\varphi'(\lambda) = \nabla f(x^k + \lambda d^k)^T d^k .$$

$$\ominus f(x^k + \lambda_k d^k) = \min_{\lambda} f(x^k + \lambda d^k)$$

→ 取得极小值的点，即 λ_k 为最优步长

$$\therefore \varphi'(\lambda_k) = \nabla f(x^k + \lambda_k d^k)^T d^k = 0 .$$

注： 因为梯度法的搜索方向 $d^{k+1} = -\nabla f(x^k + \lambda_k d^k)$, 所以

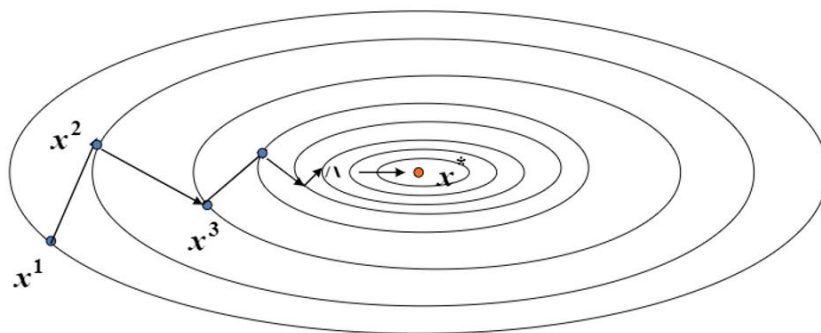
$$(d^{k+1})^T d^k = 0 \Rightarrow d^{k+1} \perp d^k .$$



4. 连续优化

• 4.2 梯度下降法

- **锯齿现象：**最速梯度下降法每次沿负梯度方向移动，即垂直于当前等值面指向函数减小最快的方向。这种正交性使得算法在接近极值点时收敛变慢，尤其是当函数曲率变化较大时，需多次迭代才能逼近最优解。



- 相邻搜索方向正交意味着算法在迭代中未利用历史搜索方向的信息，仅依赖当前梯度。**最速梯度下降法的正交性使其在高维复杂问题中收敛效率较低，仅能达到线性收敛速率。**

注 最速下降方向反映了目标函数的一种局部性质。它只是

局部目标函数值下降最快的方向。

最速下降法是线性收敛的算法。

4. 连续优化

• 4.2 梯度下降法

• 收敛性

- **性质：** 利用最速下降法搜索函数 $f(x)$ 的极小点，迭代过程中产生的序列为 $\{x^{(k)}\}_{k=0}^{\infty}$ ，如果 $\nabla f(x^{(k)}) \neq 0$ ，那么 $f(x^{(k+1)}) \leq f(x^{(k)})$

- **证明：** 已知 $x^{(k+1)} = x^{(k)} + \lambda_k d^k$ ，其中 $\lambda_k \geq 0$ 是函数 $\varphi_k(\lambda) = f(x^k + \lambda_k d^k)$ 的极小点。对于所有的 $\lambda \geq 0$ ，均有 $\varphi_k(\lambda_k) \leq \varphi_k(\lambda)$ 。

利用链式法则，可得：

$$\varphi'_k(0) = \frac{d\varphi_k}{d\lambda}(0) = \nabla f(x^k + 0d^k)^T d^k = \nabla f(x^k)^T d^k = -\|d^k\|^2 \leq 0$$

因为 $d^k = -\nabla f(x^{(k)}) \neq 0$ ，所以 $\varphi'_k(0) < 0$ ，这说明存在一个 $\bar{\lambda} > 0$ ，对于所有的 $\lambda \in (0, \bar{\lambda}]$ ，都有 $\varphi_k(0) > \varphi_k(\lambda)$ 成立。所以有：

$$f(x^{(k+1)}) = f(x^k + \lambda_k d^k) = \varphi_k(\lambda_k) \leq \varphi_k(\bar{\lambda}) < \varphi_k(0) = f(x^{(k)})$$

得证。

- **最速下降法下降特性：** 只要 $\nabla f(x^{(k)}) \neq 0$ ，就有 $f(x^{(k+1)}) \leq f(x^{(k)})$



4. 连续优化

• 4.2 梯度下降法

• 梯度下降

- 如果数据集比较小，完全可以采用全数据集(Full Batch Learning)的形式，采用全数据有两个好处：
 - 由全数据集确定的方向能够更好地代表样本总体，从而更准确地朝向极值所在的方向。
 - 由于不同权重的梯度值差别巨大，因此选取一个全局的步长（学习率）很困难。
- 但是梯度下降也存在以下缺点：
 - 梯度下降算法并不能保证被优化函数达到全局最优解，只有当损失函数为凸函数时，梯度下降算法才能保证达到全局最优解，因此不能保证一定达到全局最优，受限于损失函数是否为凸函数。
 - 梯度下降算法的另外一个问题是计算时间太长，因为要在全部训练数据上最小化损失，在海量训练数据下，这样是十分耗时。



4. 连续优化

• 4.2 梯度下降法

• 随机梯度下降 (SGD)

- 随机梯度下降算法也是称为"在线学习"。
- 这个算法优化的不是在全部训练数据上的损失函数，而是在每轮迭代中，**随机优化某一条训练数据上的损失函数**，这样每一轮参数的更新速度大大加快。
- **存在问题：**在某一条数据上损失函数更小并不代表在全部数据上的损失函数更小，于是使用随机梯度下降优化得到的神经网络甚至可能无法达到全局最优。

• 批梯度下降 (BGD)

- 为了综合梯度下降算法和随机梯度下降算法的优缺点，在实际应用中一般采用这两个算法的这种----->**每次计算一小部分训练数据的损失函数**。
- 批梯度下降算法 (Mini-batches Learning) 在深度学习很多算法的反向传播算法中非常常用。这一小部分训练数据也称为一个batch，因此也引入了batch_size的概念，batch_size顾名思义就是来度量每个batch中实例的个数。



4. 连续优化

• 4.2 梯度下降法

• 如何解决梯度下降法中无法获得全局最优步长的问题

- **自适应学习率**：使用自适应学习率算法，如AdaGrad、RMSProp或Adam，这些算法能够为每个参数调整学习率，从而更好处理不同权重的梯度差异。
- **使用随机初始化**：多次运行梯度下降法，每次使用不同初始点，然后选择最佳结果。
- **使用动量**：动量项可以帮助梯度下降法跳出局部最小值。
- **学习率衰减**：随着迭代的进行逐渐减小学习率，这样在优化过程的早期可以使用较大的学习率来快速收敛，而在后期减小学习率以进行精细调整。
- **学习率预热**：在训练的初始阶段使用较小的学习率，并逐渐增加到一个较大的值，这有助于稳定训练过程。
- **分层学习率**：为不同层的网络设置不同的学习率，这在深度学习中尤其有用，因为底层和顶层的特征可能需要不同的学习率。



4. 连续优化

• 4.3 共轭梯度法

- 从效率上来看，共轭方向法介于最速下降法与牛顿法之间。
- 从性能上来看，共轭方向法优于最速下降法，但不及牛顿法。
- 共轭梯度法具有如下特性：
 - 对于 n 维二次型问题，能够在 n 步之内得到优化结果；
 - 作为共轭方向法的典型代表，共轭梯度法不需要计算黑塞矩阵；（一阶优化）
 - 不需要存储 $n \times n$ 的矩阵，也不需要对其进行求逆运算。



4. 连续优化

• 4.3 共轭梯度法

• 共轭方向

• 几何解释

设有二次函数 $f(x) = \frac{1}{2}(x - \bar{x})^T A(x - \bar{x})$

→ 正定二次型

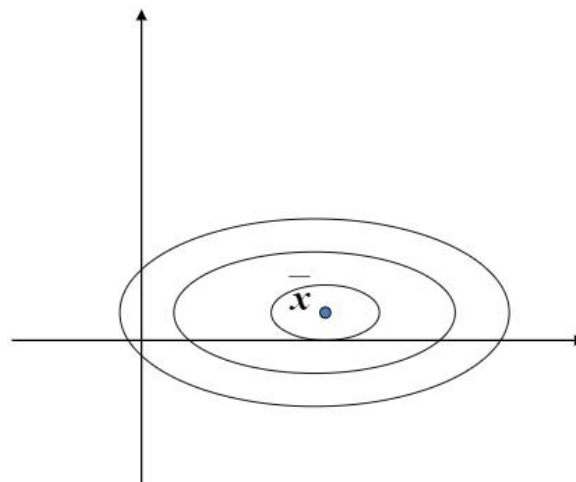
其中 A 是 $n \times n$ 对称正定矩阵,

$\bar{x}$ 是一个定点。

则函数 $f(x)$ 的等值面

$$\frac{1}{2}(x - \bar{x})^T A(x - \bar{x}) = c$$

是以 $\bar{x}$ 为中心的椭球面。



4. 连续优化

• 4.3 共轭梯度法

• 共轭方向

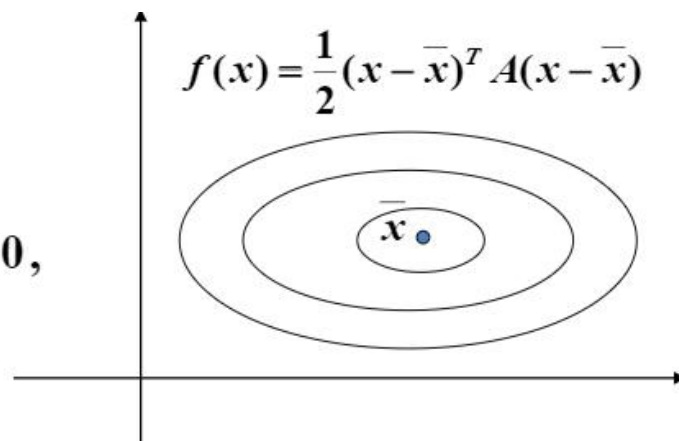
• 几何解释

由于 $\nabla f(\bar{x}) = A(x - \bar{x}) = 0$,

而 $\nabla^2 f(\bar{x}) = A$,

因为 A 正定, 所以 $\nabla^2 f(\bar{x}) = A > 0$,

因此 $\bar{x}$ 是 $f(x)$ 的极小点。



4. 连续优化

• 4.3 共轭梯度法

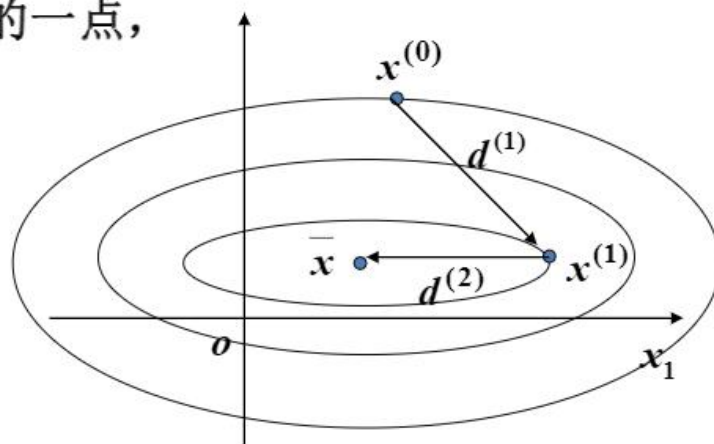
• 共轭方向

• 几何解释

设 $x^{(0)}$ 是在某个等值面上的一点,

$d^{(1)}$ 是 R^n 中的一个方向,

$x^{(0)}$ 沿着 $d^{(1)}$ 以最优步长搜索得到点 $x^{(1)}$ 。



4. 连续优化

• 4.3 共轭梯度法

• 共轭方向

• 几何解释

则 $d^{(1)}$ 是点 $x^{(1)}$ 所在等值面的切向量。

该等值面在点 $x^{(1)}$ 处的法向量为 $\nabla f(x^{(1)})$

$$\nabla f(x^{(1)}) = A(x^{(1)} - \bar{x}).$$

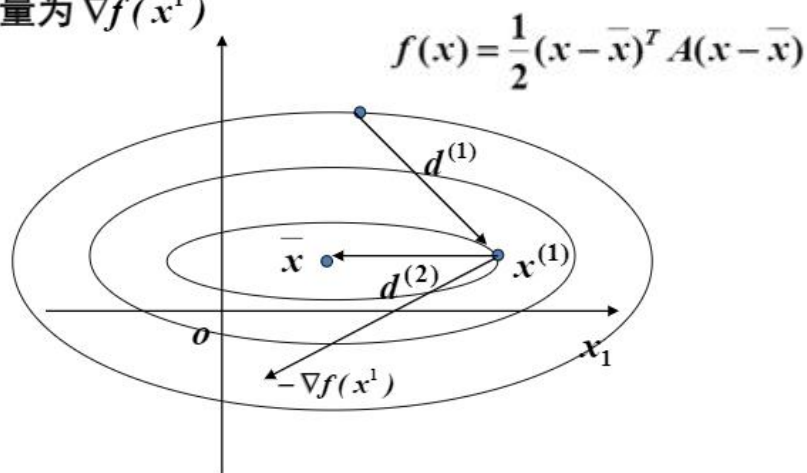
则 $d^{(1)}$ 与 $\nabla f(x^{(1)})$ 正交，

$$\text{即 } d^{(1)T} \nabla f(x^{(1)}) = 0。$$

$$\text{令 } d^{(2)} = \bar{x} - x^{(1)},$$

$$\text{所以 } d^{(1)T} A d^{(2)} = 0。$$

即等值面上一点处的切向量与由这一点指向极小点的向量关于 A 共轭。



4. 连续优化

• 4.3 共轭梯度法

• 共轭方向

• 定义

设 A 是 $n \times n$ 的对称正定矩阵, 对于 R^n 中的两个非零向量 d^1 和 d^2 ,

若有 $d^{1T} A d^2 = 0$, 则称 d^1 和 d^2 关于 A 共轭。

设 $d^1, d^2, \dots, d^k$ 是 R^n 中一组非零向量, 如果它们两两关于 A 共轭, 即 $d^{iT} A d^j = 0, i \neq j, i, j = 1, 2, \dots, k$ 。

则称这组方向是关于 A 共轭的, 也称它们是一组 A 共轭方向。

注: 如果 A 是单位矩阵, 则

$$\begin{aligned} d^{1T} \cdot I \cdot d^2 = 0 &\Rightarrow d^{1T} \cdot d^2 = 0 \\ &\Rightarrow d^1 \perp d^2 \end{aligned}$$

• 共轭是正交的推广。



4. 连续优化

• 4.3 共轭梯度法

- **定理1:** 设 A 是 n 阶对称正定矩阵, d^1, d^2, Λ, d^k 是 k 个 A 共轭的非零向量, 则这个向量组线性无关。

- **证明:** 设存在实数 $\alpha_1, \alpha_2, \Lambda, \alpha_k$, 使得

$$\sum_{i=1}^k \alpha_i d^i = 0,$$

上式两边同时左乘 $d^{jT} A$, 则有

$$\sum_{i=1}^k \alpha_i d^{jT} A d^i = 0,$$

因为 d^1, d^2, Λ, d^k 是 k 个 A 共轭的向量, 所以上式可化简为

$$\alpha_j d^{jT} A d^j = 0.$$

因为 $d^j \neq 0$, 而 A 是正定矩阵, 所以 $d^{jT} A d^j > 0$,

所以 $\alpha_j = 0, j = 1, 2, \Lambda, k$ 。

因此 d^1, d^2, Λ, d^k 线性无关。



4. 连续优化

• 4.3 共轭梯度法

定理 2. 设有函数 $f(x) = \frac{1}{2}x^T Ax + b^T x + c$,

其中 A 是 n 阶对称正定矩阵。 $d^{(1)}, d^{(2)}, \Lambda, d^{(k)}$ 是一组 A 共轭向量。

以任意的 $x^{(1)} \in R^n$ 为初始点, 依次沿 $d^{(1)}, d^{(2)}, \Lambda, d^{(k)}$ 进行搜索,

得到点 $x^{(2)}, x^{(3)}, \Lambda, x^{(k+1)}$, 则 $x^{(k+1)}$ 是函数 $f(x)$ 在 $x^{(1)} + B_k$ 上的

极小点, 其中

$$B_k = \{x | x = \sum_{i=1}^k \lambda_i d^{(i)}, \lambda_i \in R\}$$

是由 $d^{(1)}, d^{(2)}, \Lambda, d^{(k)}$ 生成的子空间。特别地, 当 $k = n$ 时, $x^{(n+1)}$ 是

$f(x)$ 在 R^n 上的唯一极小点。

推论 在上述定理条件下, 必有

$$\nabla f(x^{(k+1)})^T d^{(i)} = 0, \quad i = 1, 2, \Lambda, k。$$



4. 连续优化

求解线性方程组: $Ax = b$
等价于求 $Ax - b = 0$
等价于求 $\min \|Ax - b\|_2$

• 4.3 共轭梯度法

对于极小化问题 $\min f(x) = \frac{1}{2}x^T Ax + b^T x + c,$

其中 A 是正定矩阵, 称下述算法为共轭方向法:

- (1) 取定一组 A 共轭方向 $d^{(1)}, d^{(2)}, \dots, d^{(n)}$;
- (2) 任取初始点 $x^{(1)}$, 依次按照下式由 $x^{(k)}$ 确定点 $x^{(k+1)}$,

$$\begin{cases} x^{(k+1)} = x^{(k)} + \lambda_k d^{(k)} \\ f(x^{(k)} + \lambda_k d^{(k)}) = \min_{\lambda} f(x^{(k)} + \lambda d^{(k)}) \end{cases}$$

直到某个 $x^{(k)}$ 满足 $\nabla f(x^{(k)}) = 0$ 。

注 由定理2可知, 利用共轭方向法求解上述极小化问题, 至多经过 n 次迭代必可得到最优解。



4. 连续优化

- 4.3 共轭梯度法

- 如何选取一组共轭方向？

- 二次函数情形

- 非二次函数情形



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

• 二次函数情形

Fletcher – Reeves 共轭梯度法：

$$\min f(x) = \frac{1}{2} x^T A x + b^T x + c$$

其中 $x \in R^n$, A 是对称正定矩阵, $b \in R^n$, c 是常数。

基本思想：将共轭性和最速下降方向相结合，利用已知迭代点处的梯度方向构造一组共轭方向，并沿此方向进行搜索，求出函数的极小点。

- 共轭梯度法不需要提前给定 A 的共轭方向，而是随着迭代不断产生 A 的共轭方向。
- 每次迭代时，利用上一次的搜索方向 $d^{(k)}$ 和目标函数在当前迭代点的梯度向量 g_{k+1} 之间的线性组合上构造一个新的方向：

$$d^{(k+1)} = -g_{k+1} + \beta_k d^{(k)}$$



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

• 二次函数情形

• 以下共轭梯度算法的关键步骤：

(1) 任取初始点 $x^{(1)}$ ，第一个搜索方向取为 $d^{(1)} = -\nabla f(x^{(1)})$ ；

(2) 设已求得点 $x^{(k+1)}$ ，若 $\nabla f(x^{(k+1)}) \neq 0$ ，令 $g_{k+1} = \nabla f(x^{(k+1)})$ ，

则下一个搜索方向 $d^{(k+1)}$ 按如下方式确定：

$$\text{令 } d^{(k+1)} = -g_{k+1} + \beta_k d^{(k)} \quad (1)$$

如何确定 β_k ？

要求 $d^{(k+1)}$ 和 $d^{(k)}$ 关于 A 共轭。

则在 (1) 式两边同时左乘 $d^{(k)T} A$ ，得

$$0 = d^{(k)T} A d^{(k+1)} = -d^{(k)T} A g_{k+1} + \beta_k d^{(k)T} A d^{(k)}$$

$$\text{解得 } \beta_k = \frac{d^{(k)T} A g_{k+1}}{d^{(k)T} A d^{(k)}} \quad (2)$$



4. 连续优化

• 4.3 共轭梯度法

- **例：**利用基本的共轭算法求下列函数的极小值点。

$$f(x_1, x_2) = \frac{1}{2} x^T \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} x - x^T \begin{bmatrix} -1 \\ 1 \end{bmatrix}, x \in \mathbb{R}^2$$

初始点为 $x^0 = [0, 0]^T$ ，给定 $A = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix}$ 的共轭方向为 $d^0 = [1, 0]^T$, $d^1 = [-\frac{3}{8}, \frac{3}{4}]^T$ 。

- **解：**首先计算 $g^0 = Ax^0 - b = -b = [1, -1]^T$ ，则有

$$\beta_0 = -\frac{g^{0T} d^0}{d^{0T} A d^0} = -\frac{[1, -1] \begin{bmatrix} 1 \\ 0 \end{bmatrix}}{[1, 0] \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}} = -\frac{1}{4}$$

因此， $x^1 = x^0 + \beta_0 d^0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \frac{1}{4} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\frac{1}{4} \\ 0 \end{bmatrix}$ ，再计算

$$g^1 = Ax^1 - b = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} -\frac{1}{4} \\ 0 \end{bmatrix} - \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -\frac{3}{2} \end{bmatrix}$$

$$\beta_1 = -\frac{g^{1T} d^1}{d^{1T} A d^1} = 2$$

因此， $x^2 = x^1 + \beta_1 d^1 = \begin{bmatrix} -1 \\ \frac{3}{2} \end{bmatrix}$ 。由于函数为二次型函数，所以 $x^2 = x^*$ 。



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

• 二次函数情形

定理3 对于正定二次函数 $f(x) = \frac{1}{2}x^T Ax + b^T x + c$, FR算法在 $m \leq n$ 次一维搜索后即终止, 并且对所有的 $i (1 \leq i \leq m)$, 下列关系成立

$$(1) d^{(i)T} A d^{(j)} = 0, j = 1, 2, \Lambda, i-1;$$

$$(2) g_i^T g_j = 0, j = 1, 2, \Lambda, i-1;$$

$$(3) g_i^T d^{(i)} = -g_i^T g_i. \quad \longleftarrow \quad d^{(k+1)} = -g_{k+1} + \beta_k d^{(k)}$$

(1) 由定理3可知搜索方向 $d^{(1)}, d^{(2)}, \Lambda, d^{(m)}$ 是 A 共轭的。

(2) 算法中第一个搜索方向必须取负梯度方向, 否则构造的搜索方向不能保证共轭性。

(3) 由定理3的 (3) 可知, $g_i^T d^{(i)} = -g_i^T g_i = -\|g_i\|^2 < 0$,

所以 $d^{(i)}$ 是迭代点 $x^{(i)}$ 处的下降方向。



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

- 二次函数情形 $d^{(k+1)} = -g_{k+1} + \beta_k d^{(k)}$

(4) 由定理3, FR 算法中 β_i 的计算公式可以简化。

$$\begin{aligned}\beta_i &= \frac{d^{(i)T} A g_{i+1}}{d^{(i)T} A d^{(i)}} = \frac{g_{i+1}^T A d^{(i)}}{d^{(i)T} A d^{(i)}} \\ &= \frac{g_{i+1}^T A [(x^{(i+1)} - x^{(i)}) / \lambda_i]}{d^{(i)T} A [(x^{(i+1)} - x^{(i)}) / \lambda_i]} \quad \longleftarrow x^{(k+1)} = x^{(k)} + \lambda_k d^{(k)}\end{aligned}$$

$$\ominus \quad g_i = \nabla f(x^{(i)}) = A x^{(i)} + b.$$

$$\begin{aligned}\therefore \beta_i &= \frac{g_{i+1}^T (g_{i+1} - g_i)}{d^{(i)T} (g_{i+1} - g_i)} = \frac{\|g_{i+1}\|^2}{-d^{(i)T} g_i} \quad \longleftarrow d^{(i)T} g_{i+1} = 0 \\ &= \frac{\|g_{i+1}\|^2}{\|g_i\|^2} \quad (4)\end{aligned}$$



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

• 二次函数情形

FR算法步骤：

1. 任取初始点 $x^{(1)}$, 精度要求 ε , 令 $k=1$ 。

2. 令 $g_1 = \nabla f(x^{(1)})$, 若 $\|g_1\| < \varepsilon$, 停止, $x^{(1)}$ 为所求极小点;

否则, 令 $d^{(1)} = -g_1$, 利用公式 (3) 计算 λ_1 , 令 $x^{(2)} = x^{(1)} + \lambda_1 d^{(1)}$ 。

3. 令 $g_{k+1} = \nabla f(x^{(k+1)})$, 若 $\|g_{k+1}\| < \varepsilon$, 停止, $x^{(k+1)}$ 为所求极小点;

否则, 令 $d^{(k+1)} = -g_{k+1} + \beta_k d^{(k)}$, 其中 β_k 用公式 (4) 计算。

令 $k := k + 1$ 。

4. 利用公式 (3) 计算 λ_k , 令 $x^{(k+1)} = x^{(k)} + \lambda_k d^{(k)}$, 转3。

$$\lambda^* = -\frac{d^T \nabla f(x)}{d^T G d} \quad (3) \quad \text{--p98}$$

$$\begin{aligned} \beta_i &= \frac{g_{i+1}^T (g_{i+1} - g_i)}{d^{(i)T} (g_{i+1} - g_i)} = \frac{\|g_{i+1}\|^2}{-d^{(i)T} g_i} \\ &= \frac{\|g_{i+1}\|^2}{\|g_i\|^2} \quad (4) \quad \text{--p123} \end{aligned}$$

搜索方向

$$\begin{aligned} d^{(k+1)} &= -g_{k+1} + \beta_k d^{(k)} \\ \beta_k &= \frac{\|g_{k+1}\|^2}{\|g_k\|^2} \end{aligned}$$

搜索步长

$$\lambda^* = -\frac{d^T \nabla f(x)}{d^T G d}$$



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

• 二次函数情形

例 用FR算法求解下述问题: $\min f(x) = 2x_1^2 + x_2^2$

初始点取为 $x^{(1)} = (2, 2)^T$ 。

$$\text{解: } \ominus f(x) = \frac{1}{2}(x_1, x_2) \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \therefore A = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}.$$

$$\nabla f(x) = (4x_1, 2x_2)^T.$$

第1次迭代:

$$\text{令 } d^{(1)} = -g_1 = (-8, -4)^T,$$

$$\text{而 } \lambda_1 = -\frac{g_1^T d^{(1)}}{d^{(1)T} A d^{(1)}} = -\frac{(8, 4) \begin{bmatrix} -8 \\ -4 \end{bmatrix}}{(-8, -4) \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} -8 \\ -4 \end{bmatrix}} = \frac{5}{18}$$



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

• 二次函数情形

例 用FR算法求解下述问题: $\min f(x) = 2x_1^2 + x_2^2$

初始点取为 $x^{(1)} = (2, 2)^T$ 。

所以 $x^{(2)} = x^{(1)} + \lambda_1 d^{(1)}$

$$= (2, 2)^T + \frac{5}{18}(-8, -4)^T = \left(-\frac{2}{9}, \frac{8}{9}\right)^T$$

第2次迭代:

$$\ominus \quad g_2 = \left(-\frac{8}{9}, \frac{16}{9}\right)^T.$$

$$\therefore \beta_1 = \frac{\|g_2\|^2}{\|g_1\|^2} = \frac{\left(-\frac{8}{9}\right)^2 + \left(\frac{16}{9}\right)^2}{8^2 + 4^2} = \frac{4}{81}.$$

$$\therefore d^{(2)} = -g_2 + \beta_1 d^{(1)}$$

$$\begin{aligned} &= \left(\frac{8}{9}, -\frac{16}{9}\right)^T + \frac{4}{81}(-8, -4)^T \\ &= \frac{40}{81}(1, -4)^T \end{aligned}$$



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

• 二次函数情形

例 用FR算法求解下述问题: $\min f(x) = 2x_1^2 + x_2^2$

初始点取为 $x^{(1)} = (2, 2)^T$ 。

$$\begin{aligned}\ominus \quad \lambda_2 &= -\frac{g_2^T d^{(2)}}{d^{(2)T} A d^{(2)}} \\ &= -\frac{\frac{40}{81}(-\frac{8}{9}, \frac{16}{9}) \begin{bmatrix} 1 \\ -4 \end{bmatrix}}{(\frac{40}{81})^2 (1, -4) \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ -4 \end{bmatrix}} = \frac{9}{20}\end{aligned}$$

$$\begin{aligned}\therefore x^{(3)} &= x^{(2)} + \lambda_2 d^{(2)} \\ &= (-\frac{2}{9}, \frac{8}{9})^T + \frac{9}{20} \times \frac{40}{81} (1, -4)^T \\ &= (0, 0)^T\end{aligned}$$

$$\ominus \quad g_3 = (0, 0)^T$$

$\therefore x^{(3)}$ 即为所求极小点。



4. 连续优化

• 4.3 共轭梯度法

• 如何选取一组共轭方向？

- 非二次函数（一般函数）情形

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & x \in R^n \end{aligned}$$

对用于正定二次函数的共轭梯度法进行修改：

(1) 第一个搜索方向仍取最速下降方向，即 $d^{(1)} = -\nabla f(x^{(1)})$ 。

其它搜索方向按下式计算：

$$d^{(i+1)} = -\nabla f(x^{(i+1)}) + \beta_i d^{(i)},$$

$$\text{其中 } \beta_i = \frac{\|\nabla f(x^{(i+1)})\|^2}{\|\nabla f(x^{(i)})\|^2}。$$

(2) 搜索步长 λ_i 不能利用公式 (3) 计算，需由一维搜索确定。



4. 连续优化

• 4.3 共轭梯度法

- 共轭梯度法本质上仍为一阶方法，未利用二阶导数信息，对非二次函数的收敛性依赖梯度差的累积
- 仅依赖一阶梯度，无法捕捉函数二阶曲率信息，导致：
 - 无法区分“平坦区域”与“陡峭区域”，步长策略低效；
 - 对非凸函数易陷入鞍点或局部极小值点
- 共轭梯度法通过梯度差隐式近似二阶黑塞矩阵作用

$$\begin{aligned}\beta_i &= \frac{\mathbf{g}_{i+1}^T (\mathbf{g}_{i+1} - \mathbf{g}_i)}{\mathbf{d}^{(i)T} (\mathbf{g}_{i+1} - \mathbf{g}_i)} = \frac{\|\mathbf{g}_{i+1}\|^2}{-\mathbf{d}^{(i)T} \mathbf{g}_i} \\ &= \frac{\|\mathbf{g}_{i+1}\|^2}{\|\mathbf{g}_i\|^2} \quad (4)\end{aligned}$$

4. 连续优化

• 4.4 牛顿法

- 确定搜索方向时，如果可以引入高阶导数提高搜索效率
- 牛顿法同时利用一阶导数和二阶导数确定搜索方向，使其指向“曲率引导的最优下降路径”
- **基本思想**
 - 给定一个迭代点 $x^{(k)}$ ，首先利用泰勒展开构造二次型函数，保证构造的二次型函数与原始目标函数 $f(x)$ 在该迭代点处的一阶和二阶导数相同，并以此作为目标函数的近似表达式
 - 接下来求该近似二次型函数的极小值点，并以此点作为下次迭代的初始点
 - 重复以上过程，以求得目标函数的极小值点



4. 连续优化

• 4.4 牛顿法

• 基本思想

设目标函数是连续二阶可微的，将函数在点 $x^{(k)}$ 按泰勒级数展开，并取到二次项：

$$f(x) \approx \Phi(x^{(k)}) = f(x^{(k)}) + [\nabla f(x^{(k)})]^T (x - x^{(k)}) + \frac{1}{2} (x - x^{(k)})^T H(x^{(k)}) (x - x^{(k)})$$

对 x 求导，其极值点必满足一阶导数为零，所以，

$$\nabla \Phi(x) = \frac{\partial f(x)}{\partial x} = \nabla f(x^{(k)}) + (x - x^{(k)}) H(x^{(k)}) = 0$$

得到
$$x_{\min} = x^{(k)} - [H(x^{(k)})]^{-1} \nabla f(x^{(k)}) \quad [1]$$

式中， $[H(x^{(k)})]^{-1}$ Hessian矩阵的逆矩阵。

4. 连续优化

• 4.4 牛顿法

• 基本思想

在一般情况下, $f(x)$ 不一定是二次函数, 因而 $x_{\min}$ 也不可能是 的极值点。但是在 $x^{(k)}$ 点附近, 函数 $\Phi(x)$ 和 $f(x)$ 是近似的, 所以可以用 $x^{(k+1)}$ 点作为下一次迭代, 即得

$$x^{k+1} = x^{(k)} - [H(x^{(k)})]^{-1} \nabla f(x^{(k)}) \quad [2]$$

如果目标函数 $f(x)$ 是正定二次函数, 那么 $H(x)$ 是个正矩阵, 逼近式 [1] 是准确的。因此由 $x^{(k)}$ 点出发只要迭代一次既可以求 $f(x)$ 的极小点。

4. 连续优化

• 4.4 牛顿法

• 迭代步骤

① 给定初始点 $x^{(0)}$ ，计算精度 ε ，令 $k = 0$ ；

② 计算 $x^{(k)}$ 点的梯度 $\nabla f(x^{(k)})$ 、 $H(x^{(k)})$ 及其逆矩阵 $[H(x^{(k)})]^{-1}$ ；

③ 构造搜索方向

$$S^{(k)} = -[H(x^{(k)})]^{-1} \nabla f(x^{(k)})$$

④ 沿 $S^{(k)}$ 方向进行一维搜索，得到迭代点 $x^{(k+1)} = x^{(k)} + S^{(k)}$

➤ 收敛判断：若 $\|\nabla f(x^{(k+1)})\| \leq \varepsilon$ ，则 $x^{(k+1)}$ 为近似最优点，迭代停止，输出最优解 $x_{min} = x^{(k+1)}$ 和 $f(x_{min}) = f(x^{(k+1)})$ ，终止计算。

➤ 若不满足 $\|\nabla f(x^{(k+1)})\| \leq \varepsilon$ ，则令 $k = k + 1$ ，转到步骤2继续迭代。



4. 连续优化

• 4.4 牛顿法

- **例题：** 用牛顿法求函数 $f(x) = x_1^2 + x_2^2 - x_1x_2 - 10x_1 - 4x_2 + 60$ 的极小值。

解： (1) 取初始点 $x^{(0)} = (0,0)^T$;

(2) 计算牛顿方向：

$$\nabla f(x^{(0)}) = (2x_1 - x_2 - 10, 2x_2 - x_1 - 4)^T_{x=x^{(0)}} = (-10, -4)^T$$

$$H(x^{(0)}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \quad [H(x^{(0)})]^{-1} = \frac{1}{3} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

所以： $s^{(0)} = -[H(x^{(0)})]^{-1} \nabla f(x^{(0)})$

$$= -\frac{1}{3} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} -10 \\ -4 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 24 \\ 18 \end{bmatrix} = \begin{bmatrix} 8 \\ 6 \end{bmatrix}$$

(3) 极小值

$$x^1 = x^{(0)} + \alpha^{(0)} s^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 1 * \begin{bmatrix} 8 \\ 6 \end{bmatrix} = \begin{bmatrix} 8 \\ 6 \end{bmatrix}$$

$$\min f(x) = 8$$



4. 连续优化

• 4.4 牛顿法

• **例题：** $\min_{\mathbf{x}} f(\mathbf{x}) = x_1^2 + 2x_2^2$,

给定初始点 $\mathbf{x}^1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ ，精度参数 $\varepsilon = 10^{-6}$ 。显然，精确最优解为 $\mathbf{x}^* = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

首先， $\nabla f(\mathbf{x}) = \begin{pmatrix} 2x_1 \\ 4x_2 \end{pmatrix}$ ， $\nabla^2 f(\mathbf{x}) = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$ 正定，因此 f 是凸函数，并且 $[\nabla^2 f(\mathbf{x})]^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/4 \end{pmatrix}$ 。

$k=1$: $\nabla f(\mathbf{x}^1) = \begin{pmatrix} 2 \\ 4 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ 2 \end{pmatrix}$ ， $\|\nabla f(\mathbf{x}^1)\| = 2\sqrt{5} \geq \varepsilon$ ， $\mathbf{d}^1 = -[\nabla^2 f(\mathbf{x}^1)]^{-1} \nabla f(\mathbf{x}^1) = -\begin{pmatrix} 1/2 & 0 \\ 0 & 1/4 \end{pmatrix} \begin{pmatrix} 2 \\ 4 \end{pmatrix} =$

$\begin{pmatrix} -1 \\ -1 \end{pmatrix}$ ， $\mathbf{x}^2 = \mathbf{x}^1 + \mathbf{d}^1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ 。

$k=2$: $\nabla f(\mathbf{x}^2) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ ， $\|\nabla f(\mathbf{x}^2)\| = 0 < \varepsilon$ ，得到最优解 $\tilde{\mathbf{x}} = \mathbf{x}^2 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ 。



4. 连续优化

• 4.4 牛顿法

• 优点

- **快速局部收敛**：牛顿法利用二阶导数信息，能以二次收敛速度逼近极小值点，比仅依赖一阶导数的梯度下降方法更快。
- **精确搜索方向**：牛顿法提供的搜索方向直接指向目标函数下降最快的方向，避免了梯度下降法可能出现的“之字形”路径。
- **鲁棒性**：在目标函数曲率变化不大的区域，牛顿法表现稳定，不易受局部极小值困扰

• 缺点：

- **计算复杂度高**：牛顿法需要计算并存储黑塞矩阵及求逆，当问题规模增大时，计算和存储成本显著增加。
- **对海森矩阵性质敏感**：若海森矩阵病态或不可逆，牛顿法可能失效。在机器学习中，由于数据噪声和模型选择可能导致海森矩阵条件数大，影响算法稳定性。
- **需要合适的步长选择策略**：尽管牛顿法提供了精确的搜索方向，但实际应用中仍需通过一维搜索或其他方法确定合适的步长，以保证迭代过程的稳定性。
- **局部收敛性**：牛顿法是局部收敛的，当初始点选择不当时，可能导致不收敛。
- **对函数要求苛刻**：牛顿法要求目标函数具有连续的一、二阶偏导数，且海森矩阵必须可逆，这在实际应用中可能难以满足



4. 连续优化

• 4.5 拟牛顿法

• 基本思想：

- 牛顿法收敛很快，但需要计算黑塞矩阵的逆矩阵，而此矩阵可能是非正定的，可能导致搜索方向不是下降方向；
- 牛顿法的一大优势在于：如果初始点离极小点比较近，牛顿法可以表现出相当好的收敛性。
- 拟牛顿法使用不包含二阶导数的矩阵近似黑塞矩阵。



4. 连续优化

• 4.5 拟牛顿法

• 拟Newton条件:

- 考虑无约束非线性优化问题(UNP), 即: $\min_{\mathbf{x}} f(\mathbf{x})$, 其中 $f(x)$ 二阶连续可微。

- 设 $\mathbf{x}^k$ 是当前迭代点, $\mathbf{H} \in \mathbf{R}^{n \times n}$ 是对称矩阵, $\mathbf{B} = \mathbf{H}^{-1}$, 令

$$\mathbf{d}^k = -\mathbf{H}_k \nabla f(\mathbf{x}^k) = -\mathbf{B}_k^{-1} \nabla f(\mathbf{x}^k)$$

- 将 $\mathbf{d}^k$ 作为 f 在 $\mathbf{x}^k$ 处的搜索方向。显然, 当 $\mathbf{H}$ 是正定时, $\mathbf{d}^k$ 是 f 在 $\mathbf{x}^k$ 处的下降方向, 并且 $\mathbf{H}^{-1}$ 一范意义下和 $\mathbf{B}_k$ 一范意义下的最速下降方向, 所以可以称 $\mathbf{d}^k$ 为 f 在 $\mathbf{x}^k$ 处的 变尺度方向。



4. 连续优化

• 4.5 拟牛顿法

• 拟Newton条件:

利用梯度函数 $\nabla f(\mathbf{x})$ 的 Taylor展开式

$$\nabla f(\mathbf{x}^{k-1}) \approx \nabla f(\mathbf{x}^k) + \nabla^2 f(\mathbf{x}^k)(\mathbf{x}^{k-1} - \mathbf{x}^k)$$

即

$$\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}) \approx \nabla^2 f(\mathbf{x}^k)(\mathbf{x}^k - \mathbf{x}^{k-1})$$

记 $\mathbf{g}^k = \nabla f(\mathbf{x}^k)$, $\mathbf{p}^{k-1} = \mathbf{x}^k - \mathbf{x}^{k-1}$, $\mathbf{q}^{k-1} = \mathbf{g}^k - \mathbf{g}^{k-1}$, 则由上式,

$$\mathbf{q}^{k-1} \approx \nabla^2 f(\mathbf{x}^k)\mathbf{p}^{k-1}, \text{ 或 } [\nabla^2 f(\mathbf{x}^k)]^{-1} \mathbf{q}^{k-1} \approx \mathbf{p}^{k-1}$$

因此, 为使 H_k 是 $[\nabla^2 f(\mathbf{x}^k)]^{-1}$ 的某种近似, 要求满足

$$\mathbf{p}^{k-1} = H_k \mathbf{q}^{k-1}$$

- 称为**关于 H_k 的拟Newton条件**, 刻画了 H_k 近似于 $[\nabla^2 f(\mathbf{x}^k)]^{-1}$ 时应具有的一个重要特性。

4. 连续优化

• 4.5 拟牛顿法

• 拟Newton条件:

- 由上式可知, B_k 应满足: $q^{k-1} = B_k p^{k-1}$
- 同样, 上式称为关于 B_k 的拟Newton条件, 此时
$$d^k = -H_k \nabla f(x^k) = -B_k^{-1} \nabla f(x^k)$$
称为 f 在 x^k 处的拟Newton方向。

- 其实, 拟Newton条件使二次函数

$$\tilde{f}(x) = f(x^k) + \nabla f(x^k)^T (x - x^k) + \frac{1}{2} (x - x^k)^T B_k (x - x^k)$$

具有插值性质:

$$\tilde{f}(x^k) = f(x^k), \quad \nabla \tilde{f}(x^k) = \nabla f(x^k), \quad \nabla \tilde{f}(x^{k-1}) = \nabla f(x^{k-1})$$

4. 连续优化

• 4.5 拟牛顿法

设已有对称正定矩阵 H_k , 令 $H_{k+1} = H_k + \Delta H_k$, 其中 ΔH_k 称为修正矩阵。由上面公式知, ΔH_k 应满足条件 $p^k = H_k q^k + \Delta H_k q^k$ 即

$$\Delta H_k q^k = p^k - H_k q^k$$

$$p^{k-1} = H_k q^{k-1}$$

同理:

ΔB 应满足条件 $B_k p^k + \Delta B_k p^k = q^k$, 即

$$q^{k-1} = B_k p^{k-1}$$

$$\Delta B_k p^k = q^k - B_k p^k$$



4. 连续优化

• 4.5 拟牛顿法

• DFP法- H_k 的秩2修正法

- 该方法首次由Davidon于1959年提出，后又由Feltcher和Powell于1963年进行改进和简化，故称为DFP法。
- DFP方法基本思想：构造一个秩为2的对称修正矩阵 ΔH_k ，使它满足条件：

$$\Delta H_k q^k = p^k - H_k q^k$$

- 令 $\Delta H_k = \alpha u u^T + \beta v v^T$

其中 $\alpha, \beta \in R^1$ 和 $u, v \in R^n$ 为待定量。将(4.2.1)代入(4.1.4)得，

$$\alpha(u^T q^k)u + \beta(v^T q^k)v = p^k - H_k q^k$$

4. 连续优化

• 4.5 拟牛顿法

• DFP法- H_k 的秩2修正法

显然，我们可使 $\alpha, \beta \in R^1$ 和 $u, v \in R^n$ 满足条件

$$\alpha(u^T q^k) = 1, \quad u = p^k, \quad \beta(v^T q^k) = -1, \quad v = H_k q^k$$

即 $\alpha = \frac{1}{u^T q^k} = \frac{1}{(p^k)^T q^k}$, $\beta = -\frac{1}{v^T q^k} = -\frac{1}{(q^k)^T H_k q^k}$, 代入(4.2.1)得修正矩阵 $\Delta H_k = \alpha u u^T + \beta v v^T$

$$\Delta H_k = \frac{p^k (p^k)^T}{(p^k)^T q^k} - \frac{H_k q^k (q^k)^T H_k}{(q^k)^T H_k q^k}$$

因此关于 H_k 的递推公式为

$$H_{k+1} = H_k + \frac{p^k (p^k)^T}{(p^k)^T q^k} - \frac{H_k q^k (q^k)^T H_k}{(q^k)^T H_k q^k}$$



4. 连续优化

• 4.5 拟牛顿法

• DFP法- H_k 的秩2修正法

下面的定理告诉我们, 当 H_k 为对称正定矩阵时, 只要步长 λ_k 满足一定的条件, 就能保证 H_{k+1} 的对称正定性, 从而就能保证搜索方向 $d^{k+1} = -H\nabla f(\mathbf{x}^{k+1})$ 的下降性。

设 H_k 是对称正定矩阵, $\nabla f(\mathbf{x}^k) \neq \mathbf{0}$, λ_k 满足 Wolfe-Powell 准则, 使 $\varphi'(\lambda_k) \geq \sigma_3 \varphi'(0)$,

$$\nabla f(\mathbf{x}^k + \lambda_k \mathbf{d}^k)^T \mathbf{d}^k = \nabla f(\mathbf{x}^{k+1})^T \mathbf{d}^k \geq \sigma_3 \nabla f(\mathbf{x}^k)^T \mathbf{d}^k$$

其中 $0 < \sigma_3 < 1$, 则由 DFP 公式确定的 H_{k+1} 也是对称正定矩阵。



4. 连续优化

• 4.5 拟牛顿法

• 求解UNP的DFP方法:

1. 初始步。给定初始点 $\mathbf{x}^1 \in R^n$, 初始正定矩阵 H_1 (例如取 H_1 为单位阵), 精度参数 $\varepsilon > 0$ 。若 $\|\nabla f(\mathbf{x}^1)\| \leq \varepsilon$, 停止, 得 $\tilde{\mathbf{x}} = \mathbf{x}^1$, 否则令 $\mathbf{g}^1 = \nabla f(\mathbf{x}^1)$, $k=1$ 。

2. 构造搜索方向。令 $\mathbf{d}^k = -H_k \mathbf{g}^k$ 。

3. 进行一维搜索。求 $\min_{\lambda \geq 0} f(\mathbf{x}^k + \lambda \mathbf{d}^k)$ 的最优步长或满足 Wolfe-Powell 准则的可接受步长 λ_k , 使

$\phi'(\lambda_k) \geq \sigma_3 \phi'(0)$, 使

$$\nabla f(\mathbf{x}^k + \lambda_k \mathbf{d}^k)^T \mathbf{d}^k = \nabla f(\mathbf{x}^{k+1})^T \mathbf{d}^k \geq \sigma_3 (\mathbf{g}^k)^T \mathbf{d}^k$$

其中 $0 < \sigma_3 < 1$ 。令 $\mathbf{x}^{k+1} = \mathbf{x}^k + \lambda_k \mathbf{d}^k$, $\mathbf{g}^{k+1} = \nabla f(\mathbf{x}^{k+1})$ 。

4. 终止判别。若 $\|\mathbf{g}^{k+1}\| \leq \varepsilon$, 停止, 得 $\tilde{\mathbf{x}} = \mathbf{x}^{k+1}$, 否则转 5。

5. 校正正定矩阵。若 $k=n$, 则令 $H_{k+1} = H_1$, 否则令 $\mathbf{p}^k = \mathbf{x}^{k+1} - \mathbf{x}^k$, $\mathbf{q}^k = \mathbf{g}^{k+1} - \mathbf{g}^k$, 由 DFP 公式求得 H_{k+1} 。

令 $k=k+1$, 转 2。

$$H_{k+1} = H_k + \frac{\mathbf{p}^k (\mathbf{p}^k)^T}{(\mathbf{p}^k)^T \mathbf{q}^k} - \frac{H_k \mathbf{q}^k (\mathbf{q}^k)^T H_k}{(\mathbf{q}^k)^T H_k \mathbf{q}^k}$$



4. 连续优化

- 4.5 拟牛顿法

- 求解UNP的DFP方法：

- 例： 用 DFP 法求解无约束优化问题

$$\min f(\mathbf{x}) = x_1 + 2x_2$$

给定初始点 $\mathbf{x}^1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, 初始矩阵 $H_1 = I$, 精度参数 $\varepsilon = 10^{-6}$,
并采用最优步长。



4. 连续优化

• 4.5 拟牛顿法

• 求解UNP的DFP方法：

首先, $\nabla f(\mathbf{x}) = \begin{pmatrix} 2x_1 \\ 4x_2 \end{pmatrix}$, $H = \nabla^2 f(\mathbf{x}) = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$ 。

$k=1$: $\nabla f(\mathbf{x}^1) = \begin{pmatrix} 2 \\ 4 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\|\nabla f(\mathbf{x}^1)\| = 2\sqrt{5} \geq \varepsilon$, $\mathbf{d}^1 = -H_1 \nabla f(\mathbf{x}^1) = -2 \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, 取 $\mathbf{d}^1 = -\begin{pmatrix} 1 \\ 2 \end{pmatrix}$,

$\lambda_1 = -\frac{\nabla f(\mathbf{x}^1)^T \mathbf{d}^1}{(\mathbf{d}^1)^T H \mathbf{d}^1} = \frac{5}{9}$, $\mathbf{x}^2 = \mathbf{x}^1 + \lambda_1 \mathbf{d}^1 = \begin{pmatrix} 1-5/9 \\ 1-10/9 \end{pmatrix} = \begin{pmatrix} 4/9 \\ -1/9 \end{pmatrix}$ 。 $\nabla f(\mathbf{x}^2) = \begin{pmatrix} 8/9 \\ -4/9 \end{pmatrix} = \frac{4}{9} \begin{pmatrix} 2 \\ -1 \end{pmatrix}$,

$\|\nabla f(\mathbf{x}^2)\| = \frac{4}{9}\sqrt{5} \geq \varepsilon$ 。 $\mathbf{p}^1 = \mathbf{x}^2 - \mathbf{x}^1 = -\frac{5}{9} \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\mathbf{q}^1 = \nabla f(\mathbf{x}^2) - \nabla f(\mathbf{x}^1) = -\frac{10}{9} \begin{pmatrix} 1 \\ 4 \end{pmatrix}$,

$H_2 = H_1 + \frac{\mathbf{p}^1 (\mathbf{p}^1)^T}{(\mathbf{p}^1)^T \mathbf{q}^1} - \frac{H_1 \mathbf{q}^1 (\mathbf{q}^1)^T H_1}{(\mathbf{q}^1)^T H_1 \mathbf{q}^1} = \frac{1}{342} \begin{pmatrix} 305 & -38 \\ -38 & 86 \end{pmatrix}$ 。

$k=2$: $\mathbf{d}^2 = -H_2 \nabla f(\mathbf{x}^2) = -\frac{1}{342} \cdot \frac{4}{9} \begin{pmatrix} 648 \\ -162 \end{pmatrix}$, 故取 $\mathbf{d}^2 = \begin{pmatrix} -4 \\ 1 \end{pmatrix}$, $\lambda_2 = -\frac{\nabla f(\mathbf{x}^2)^T \mathbf{d}^2}{(\mathbf{d}^2)^T H \mathbf{d}^2} = \frac{1}{9}$, $\mathbf{x}^3 = \mathbf{x}^2 + \lambda_2 \mathbf{d}^2 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$,

$\nabla f(\mathbf{x}^3) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, 得到最优解 $\tilde{\mathbf{x}} = \mathbf{x}^3 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ 。



4. 连续优化

• 4.5 拟牛顿法

• BFGS方法-DFP法的对偶方法:

$$B_{k+1} = B_k + \frac{q^k (q^k)^T}{(p^k)^T q^k} - \frac{B_k p^k (p^k)^T B_k}{(p^k)^T B_k p^k}$$

根据逆的秩 1 校正公式:

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1}u} \quad (A \text{ 可逆}, 1 + v^T A^{-1}u \neq 0)$$

由此得到 $H_k = B_k^{-1}$ 的递推公式为

$$H_{k+1} = H_k + (1 + \frac{(q^k)^T H_k q^k}{(p^k)^T q^k}) \frac{p^k (p^k)^T}{(p^k)^T q^k} - \frac{p^k (q^k)^T H_k + H_k q^k (p^k)^T}{(p^k)^T q^k}$$



4. 连续优化

• 4.5 拟牛顿法

• BFGS方法-DFP法的对偶方法：

1. 在求解(UNP)的DFP 算法的第5 步中用 “BFGS 公式确定 H_{k+1} ” 代替 “DFP 公式确定 H_{k+1} ”，即得到BFGS 算法。
2. BFGS 公式满足拟Newton 条件，具有正定性、共轭性和遗传性，因此BFGS 法具有二次终止性。
3. 对于一般的无约束非线性规划问题(UNP)，在一定条件下，BFGS 法具有超线性收敛速度. 并且，BFGS 算法的计算效果优于DFP 算法。



寄语

以矩阵思维解构人生命题：

- **基向量与生命秩维**

人生是向量空间的组合轨迹，教育、职业等关键选择构成「基向量」，其线性组合决定生命形态。需锚定核心基向量，避免冗余选择导致「秩亏」——如同矩阵秩不足会丧失维度完整性，精力过度分散将使生命价值坍缩。

- **特征值与核心能力主轴**

个人核心能力（如创造力、韧性）是人生矩阵的「特征值」，发展方向为「特征向量」。主特征值决定轨迹主轴，即便外界环境（矩阵变换）更迭，保持特征值优势方能在人生映射中维持方向稳定性。

- **矩阵分解的生命启示**

- **SVD 降维法则：**人生经历可借奇异值分解筛滤本质——以核心价值（奇异值）为锚，剥离冗余经验（非主奇异向量），使生命矩阵在降维后仍保有意义内核。
- **QR 正交平衡术：**将人生拆分为正交矩阵 Q （方向维度）与上三角矩阵 R （递进路径）：事业、健康等维度需如 Q 的正交列向量般独立平衡（不互相消耗），方能沿 R 的阶梯轨迹持续攀升。

•



寄语

以最优优化思维解析人生抉择：

- **目标迭代策略：**人生目标类似函数求最小值，需沿“反思错误”的方向持续调整（如最速梯度法）。行动强度需平衡“激进试错”与“稳健调整”——过度冲动易偏离方向，过度犹豫则进步迟缓。
- **心态构建逻辑：**非凸优化中易陷入“局部最优”（如挫折后的自我设限），而“凸性心态”需将挫折视为暂时波动，以长期学习拓展可能性，避免被短期困境困住。
- **视角转换技巧：**人生难题的“对偶视角”常藏破局关键。例如职业选择中，从“能力换资源”转向“资源反哺能力”，如同线性规划的对偶问题，换角度可发现隐藏最优解。
- **关键节点决策：**人生转折类似函数鞍点，需评估长远影响：若风险可控（如创业初期），可果断前进；若存在不确定性，需用“经验试探法”逐步验证方向，避免盲目决策。



5. 图论与离散优化

- 常见的图论问题
- 最短路径问题
- 旅行商问题
- 最小割最大流



5. 图论与离散优化

• 5.1 常见的图论问题

- **图论**是组合数学的一个分支，已有二百六十多年的发展历史，在漫长的发展过程中，产生了一些经典问题：

- 哥尼斯堡七桥问题
- 四色问题
- Hamilton问题
- 旅行商问题

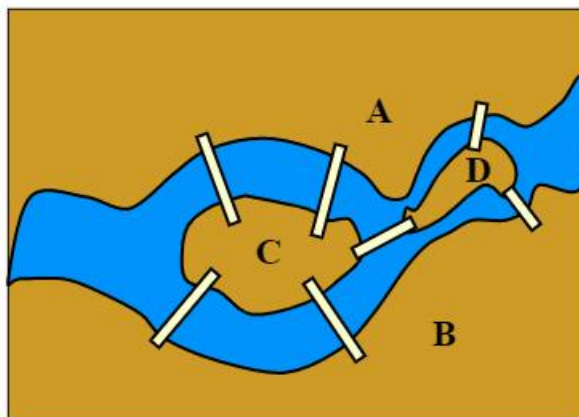


5. 图论与离散优化

• 5.1 常见的图论问题

• 哥尼斯堡七桥问题：

- 哥尼斯堡七桥位于前苏联的加里宁格勒



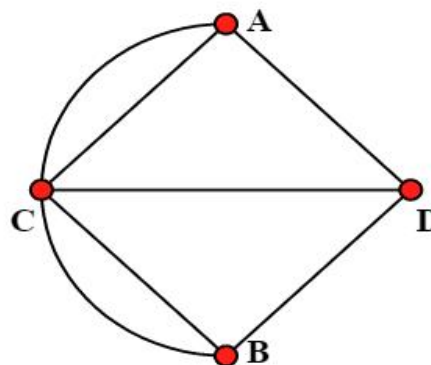
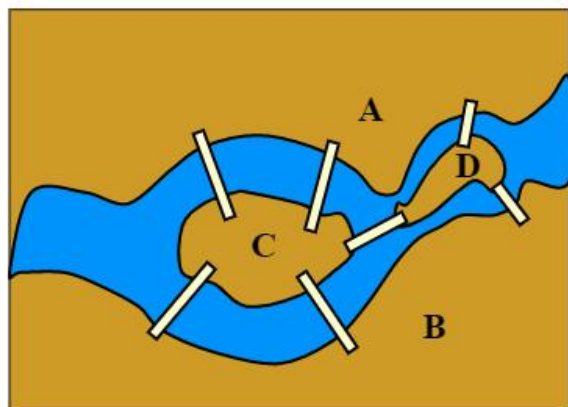
河中有两个小岛C与D，并有七座桥连接岛与河岸及岛与岛。是否存在一种走法，从四块陆地中的任意一块开始，通过每一座桥恰好一次再回到起点。

- 哥尼斯堡七桥问题是图论的起源

5. 图论与离散优化

• 5.1 常见的图论问题

• 哥尼斯堡七桥问题：

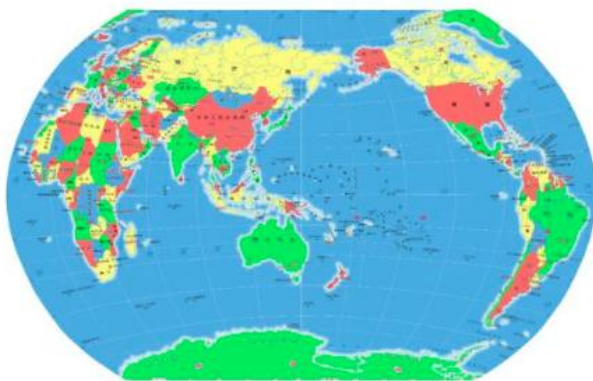


- 1736年欧拉把陆地缩为一点，把桥作为连接点的边，将这个问题抽象一笔画问题。即能否从某一点开始不重复地一笔画出这个图形，最终回到原点。
- 欧拉证明了这是不可能的，因为这个图形中每一个顶点都与奇数条边相连接，不可能将它一笔画出。

5. 图论与离散优化

- 5.1 常见的图论问题

- 四色问题：

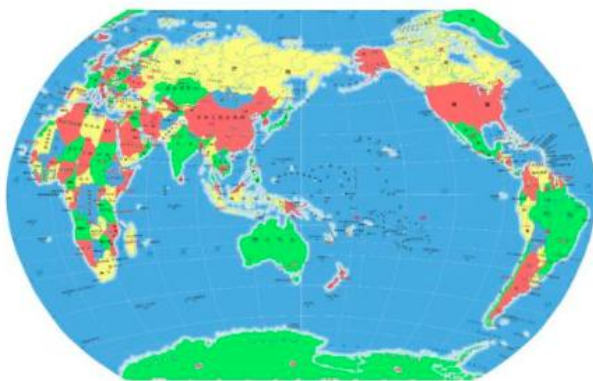


为了能够迅速地区分球面地图上的各个国家（假设这些国家在地图上都是连通的）。需要用若干种颜色对这些国家着色，要求具有公共边界的两个国家涂染不同的颜色。那么最少需要多少种颜色？

5. 图论与离散优化

• 5.1 常见的图论问题

• 四色问题：

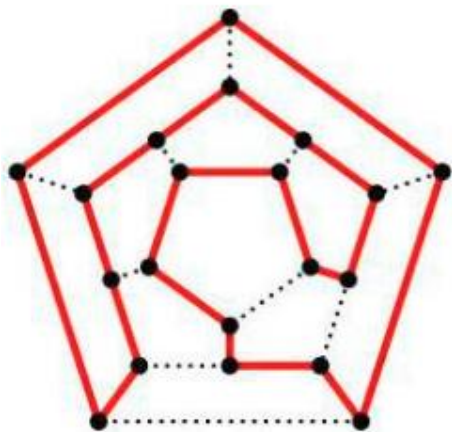


- 1852年，毕业于伦敦大学的格斯里 (Francis Guthrie) 来到一家科研单位搞地图着色工作时，发现每幅地图都可以只用四种颜色着色。
- 1976年6月，在美国伊利诺斯大学的两台不同的电子计算机上，用了1200个小时，作了100亿个判断，结果没有一张地图是需要五色的，最终证明了四色定理

5. 图论与离散优化

• 5.1 常见的图论问题

• Hamilton问题:



与著名的七桥问题不同:

过桥只需要确定起点, 而不用确定终点。哈密顿问题寻找一条从给定的起点到给定的终点沿途恰好经过所有其他城市一次的路径。

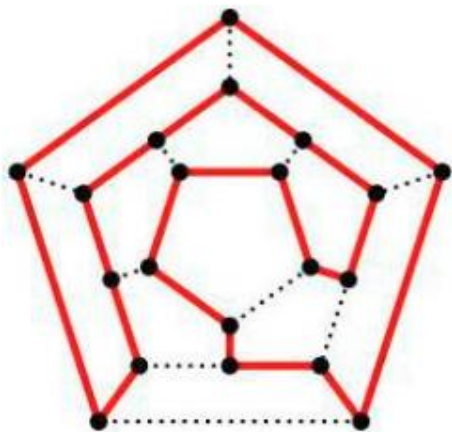
哈密顿图:

是一个无向图, 由天文学家哈密顿提出, 由指定的起点前往指定的终点, 途中经过所有其他节点且只经过一次。闭合的哈密顿路径称作哈密顿回路含有图中所有顶点的路径称作哈密顿路径。

5. 图论与离散优化

• 5.1 常见的图论问题

• Hamilton问题:



与著名的七桥问题不同:

过桥只需要确定起点, 而不用确定终点。哈密顿问题寻找一条从给定的起点到给定的终点沿途恰好经过所有其他城市一次的路径。

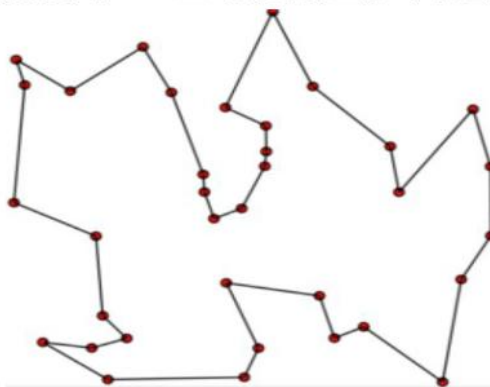
- 哈密顿路径问题在上世纪七十年代初, 终于被证明是“NP 完备”的。
- 实际上对于某些顶点数不到100的网络, 利用现有最好的算法和计算机也需要比较荒唐的时间(比如几百年)才能确定其是否存在一条这样的路径。

5. 图论与离散优化

• 5.1 常见的图论问题

• 旅行商问题：

TSP问题 (Traveling Salesman Problem) 由威廉哈密顿爵士和英国数学家克克曼于19世纪初提出。问题描述如下：



有若干个城市，任何两个城市之间的距离都是确定的，现要求一旅行商从某城市出发必须经过每一个城市且只在一个城市逗留一次，最后回到出发的城市，问如何事先确定一条最短的线路已保证其旅行的费用最少？

5. 图论与离散优化

• 5.1 常见的图论问题

• 旅行商问题：

$w(a,b)$	Shanghai	Hefei	Guangzhou	Chengdu
Beijing	1244 km	1044 km	2174 km	1854 km
Chengdu	2095 km	1615 km	1954 km	
Guangzhou	1529 km	1257 km		
Hefei	472 km			



旅行推销员问题是图论中最著名的问题之一，即“已给一个 n 个点的完全图，每条边都有一个长度，求总长度最短的经过每个顶点正好一次的封闭回路”。Edmonds, Cook和Karp等人发现，这批难题有一个值得注意的性质，对其中一个问题存在有效算法时，每个问题都会有有效算法。

5. 图论与离散优化

• 5.2 最短路径问题

带权图与最短通路问题

- **带权图**：三元组 (V, E, W) , (V, E) 是图, W 是从 E 到非负实数集的一个函数。 $W(e)$ 表示边 e 的权。
- 一条通路上所有边的权的和称为该通路的长度。
- 两点之间长度**最小**的通路称为两点之间的最短通路, 不一定是唯一的。
- **单源点最短路径问题**

给定带权图 $G(V, E, W)$ 并指定一个源点, 确定该源点到图中其它任一顶点的最短路 (长度和路径) 。



5. 图论与离散优化

• 5.2 最短路径问题

Dijkstra最短路径的算法思想(1959)

- 源点s到顶点v的最短路径若为s...uv, 则s...u是s到u的最短路径。
- (n-1)条最短路径按照其长度的非减次序求得, 设它们的相应端点分别为 $u_1, \dots, u_{n-1}$, 最短路径长度记为 $d(s, u_i)$, $i=1, \dots, n-1$.
- 假设前i条最短路径已知, 第(i+1)条最短路径长度:

$$d(s, u_{i+1}) = \min\{d(s, u_j) + W(u_j, u_{i+1}) \mid j=1, \dots, i\}$$



5. 图论与离散优化

• 5.2 最短路径问题

求最短路径的Dijkstra算法

- 输入：连通带权图 G , $|V_G|=n$, 指定顶点 $s \in V_G$
- 输出：每个顶点 v 的标注 $(L(v), u)$, 其中：
 - $L(v)$ 即从 s 到 v 的最短路径长度（目前可得的）
 - u 是该路径上 v 前一个顶点。



5. 图论与离散优化

• 5.2 最短路径问题

Dijkstra算法的描述

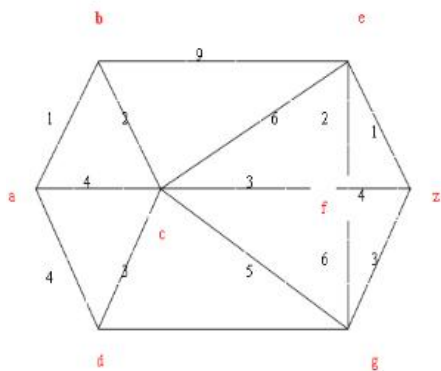
1. 初始化: $i=0$, $S_0=\{s\}$, $L(s)=0$, 对其它一切 $v \in V_G$, 将 $L(v)$ 置为 ∞ 。
若 $n=1$, 结束。
2. $\forall v \in S_i' = V_G - S_i$, 比较 $L(v)$ 和 $L(u_i) + W(u_i, v)$ 的值 ($u_i \in S_i$)
如果 $L(u_i) + W(u_i, v) < L(v)$, 则将 v 的标注更新为 $(L(u_i) + W(u_i, v), u_i)$,
即: $L(v) = \min\{L(v), \min_{u \in S_i}\{L(u) + W(u, v)\}\}$
3. 对所有 S_i' 中的顶点, 找出具有最小 $L(v)$ 的顶点 v , 作为 u_{i+1}
4. $S_{i+1} = S_i \cup \{u_{i+1}\}$
5. $i = i+1$; 若 $i=n-1$, 终止。否则: 转到第2步。



5. 图论与离散优化

• 5.2 最短路径问题

求最短路的一个例子



初始情形:

$$S = \{a\}, \bar{S} = \{b, c, d, e, f, g, z\};$$

$$\text{dist}[a] = 0, \text{dist}[b] = 1, \text{dist}[c] = 4, \text{dist}[d] = 4,$$

$$\text{dist}[e] = \text{dist}[f] = \text{dist}[g] = \text{dist}[z] = \infty;$$

$$\text{prev}[b] = \text{prev}[c] = \text{prev}[d] = \text{prev}[e]$$

$$= \text{prev}[f] = \text{prev}[g] = \text{prev}[h] = a;$$

$$u = a;$$

此时，我们发现对于所有 $v \in \bar{S}$ ， $\text{dist}[v]$ 最小的是 $\text{dist}[b] = 1$
则此时肯定起点 a 到 b 的最短路已经确定：
即是 $a \rightarrow b$ ，权和为 1

5. 图论与离散优化

• 5.2 最短路径问题

此时, 开始第一轮迭代:

$$S = \{a, b\}, \bar{S} = \{c, d, e, f, g, z\}, u \leftarrow b$$

$$\text{dist}[a] = 0, \text{dist}[b] = 1,$$

$$\text{dist}[c] = \min\{\text{dist}[c], \text{dist}[u] + w(u, c)\} = \min\{4, 3\} = 3$$

$$\text{dist}[d] = \min\{\text{dist}[d], \text{dist}[u] + w(u, d)\} = \min\{4, \infty\} = 4,$$

$$\text{dist}[e] = \min\{\text{dist}[e], \text{dist}[u] + w(u, e)\} = \min\{\infty, 10\} = 10$$

$$\text{dist}[f] = \min\{\text{dist}[f], \text{dist}[u] + w(u, f)\} = \min\{\infty, \infty\} = \infty$$

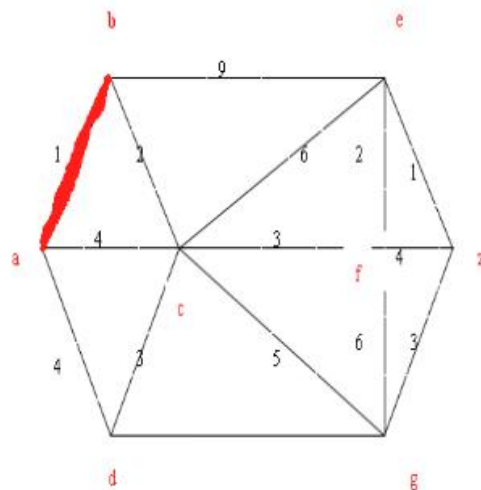
$$\text{dist}[g] = \min\{\text{dist}[g], \text{dist}[u] + w(u, g)\} = \min\{\infty, \infty\} = \infty$$

$$\text{dist}[z] = \min\{\text{dist}[z], \text{dist}[u] + w(u, z)\} = \min\{\infty, \infty\} = \infty;$$

$$\text{prev}[c] = u = b, \text{prev}[e] = u = b,$$

$$\text{prev}[b] = \text{prev}[d] = \text{prev}[f] = \text{prev}[g] = \text{prev}[z] = a;$$

a ---> b ---> c



5. 图论与离散优化

• 5.2 最短路径问题

此时, 开始第二轮迭代:

$$S = \{a, b, c\}, \bar{S} = \{d, e, f, g, z\}, u \leftarrow c$$

$$\text{dist}[a] = 0, \text{dist}[b] = 1, \text{dist}[c] = 3$$

$$\text{dist}[d] = \min \{ \text{dist}[d], \text{dist}[u] + w(u, d) \} = \min \{ 4, 6 \} = 4,$$

$$\text{dist}[e] = \min \{ \text{dist}[e], \text{dist}[u] + w(u, e) \} = \min \{ 10, 9 \} = 9$$

$$\text{dist}[f] = \min \{ \text{dist}[f], \text{dist}[u] + w(u, f) \} = \min \{ \infty, 6 \} = 6$$

$$\text{dist}[g] = \min \{ \text{dist}[g], \text{dist}[u] + w(u, g) \} = \min \{ \infty, 8 \} = 8$$

$$\text{dist}[z] = \min \{ \text{dist}[z], \text{dist}[u] + w(u, z) \} = \min \{ \infty, \infty \} = \infty;$$

$$\text{prev}[e] = u = c, \text{prev}[f] = \text{prev}[g] = c, \text{prev}[c] = b;$$

$$\text{prev}[b] = \text{prev}[d] = \text{prev}[z] = a;$$

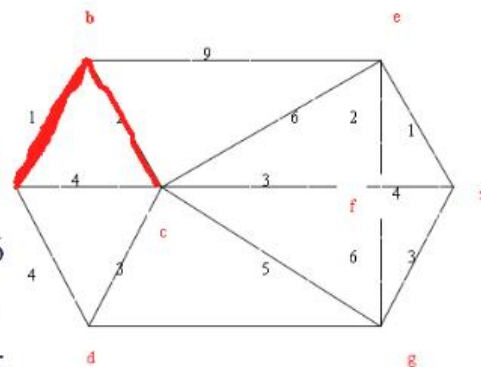
此时, $a \rightarrow e$ 的当前最短路更新为 $a \rightarrow b \rightarrow c \rightarrow e$

$a \rightarrow f$ 的当前最短路更新为 $a \rightarrow b \rightarrow c \rightarrow f$

$a \rightarrow g$ 的当前最短路更新为 $a \rightarrow b \rightarrow c \rightarrow g$ 。

并且当前 $\bar{S}$ 中最短的 dist 是 $\text{dist}[d] = 4$ 则 $a \rightarrow d$ 的最短路已经确定即是 $a \rightarrow d$

下一轮迭代时, $\bar{S} = \{e, f, g, z\}$



5. 图论与离散优化

• 5.2 最短路径问题

此时, 开始第三轮迭代:

$$S = \{a, b, c, d\}, \bar{S} = \{e, f, g, z\}, u \leftarrow d$$

$$\text{dist}[a] = 0, \text{dist}[b] = 1, \text{dist}[c] = 3, \text{dist}[d] = 4$$

$$\text{dist}[e] = \min\{\text{dist}[e], \text{dist}[u] + w(u, e)\} = \min\{9, \infty\} = 9$$

$$\text{dist}[f] = \min\{\text{dist}[f], \text{dist}[u] + w(u, f)\} = \min\{6, \infty\} = 6$$

$$\text{dist}[g] = \min\{\text{dist}[g], \text{dist}[u] + w(u, g)\} = \min\{8, 11\} = 8$$

$$\text{dist}[z] = \min\{\text{dist}[z], \text{dist}[u] + w(u, z)\} = \min\{\infty, \infty\} = \infty;$$

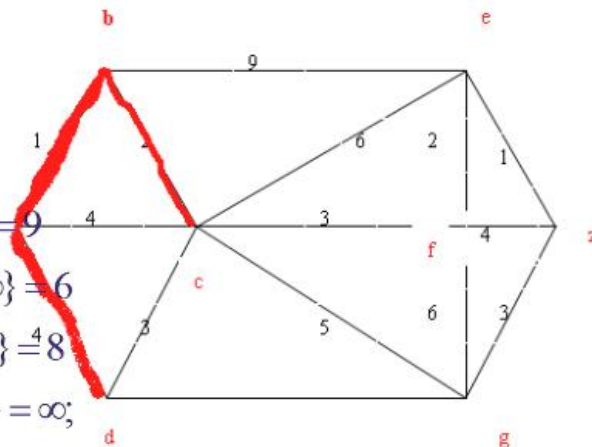
$$\text{prev}[e] = c, \text{prev}[f] = \text{prev}[g] = c, \text{prev}[c] = b;$$

$$\text{prev}[b] = \text{prev}[d] = \text{prev}[z] = a;$$

此时没有道路被更新

并且当前 $\bar{S}$ 中最短的 dist 是 $\text{dist}[f] = 6$, 则 $a \rightarrow f$ 的最短路已经确定即是 $a \rightarrow b \rightarrow c \rightarrow f$

下一轮迭代时, $\bar{S} = \{e, g, z\}$



5. 图论与离散优化

• 5.2 最短路径问题

此时, 开始第四轮迭代:

$$S = \{a, b, c, d, f\}, \bar{S} = \{e, g, z\}, u \leftarrow f$$

$$\text{dist}[a] = 0, \text{dist}[b] = 1, \text{dist}[c] = 3, \text{dist}[d] = 4, \text{dist}[f] = 6$$

$$\text{dist}[e] = \min\{\text{dist}[e], \text{dist}[u] + w(u, e)\} = \min\{9, 8\} = 8$$

$$\text{dist}[g] = \min\{\text{dist}[g], \text{dist}[u] + w(u, g)\} = \min\{8, 12\} = 8$$

$$\text{dist}[z] = \min\{\text{dist}[z], \text{dist}[u] + w(u, z)\} = \min\{\infty, 10\} = \infty;_4$$

$$\text{prev}[e] = u = f, \text{prev}[f] = \text{prev}[g] = c, \text{prev}[c] = b;$$

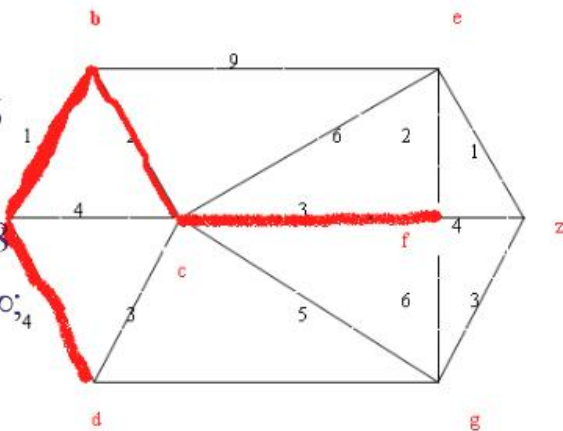
$$\text{prev}[b] = \text{prev}[d] = a, \text{prev}[z] = f$$

此时 $a \rightarrow e$ 道路被更新为 $a \rightarrow b \rightarrow c \rightarrow f \rightarrow e$

$a \rightarrow z$ 道路被更新为 $a \rightarrow b \rightarrow c \rightarrow f \rightarrow z$

并且当前 $\bar{S}$ 中最短的 dist 是 $\text{dist}[e] = 8$, 则 $a \rightarrow e$ 的最短路已经确定即是 $a \rightarrow b \rightarrow c \rightarrow f \rightarrow e$

下一轮迭代时, $\bar{S} = \{g, z\}$



5. 图论与离散优化

• 5.2 最短路径问题

此时, 开始第五轮迭代:

$$S = \{a, b, c, d, f, e\}, \bar{S} = \{g, z\}, u \leftarrow e$$

$$\text{dist}[a] = 0, \text{dist}[b] = 1, \text{dist}[c] = 3, \text{dist}[d] = 4, \text{dist}[f] = 6$$

$$\text{dist}[e] = 8$$

$$\text{dist}[g] = \min \{ \text{dist}[g], \text{dist}[u] + w(u, g) \} = \min \{ 8, \infty \} = 8$$

$$\text{dist}[z] = \min \{ \text{dist}[z], \text{dist}[u] + w(u, z) \} = \min \{ 10, 9 \} = 9;$$

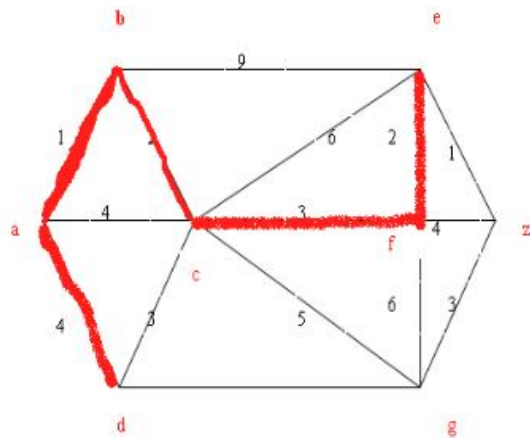
$$\text{prev}[e] = f, \text{prev}[f] = \text{prev}[g] = c, \text{prev}[c] = b;$$

$$\text{prev}[b] = \text{prev}[d] = a, \text{prev}[z] = e$$

此时 $a \rightarrow z$ 道路被更新为 $a \rightarrow b \rightarrow c \rightarrow f \rightarrow e \rightarrow z$

并且当前 $\bar{S}$ 中最短的 dist 是 $\text{dist}[g] = 8$ 则 $a \rightarrow g$ 的最短路已经确定即是 $a \rightarrow b \rightarrow c \rightarrow g$

下一轮迭代时, $\bar{S} = \{z\}$



5. 图论与离散优化

• 5.2 最短路径问题

此时, 开始第六轮迭代:

$S = \{a, b, c, d, f, e, g\}, \bar{S} = \{z\}, u \leftarrow g$

$dist[a]=0, dist[b]=1, dist[c]=3, dist[d]=4, dist[f]=6$

$dist[e]=8, dist[g]=8$

$dist[z] = \min\{dist[z], dist[u] + w(u, z)\} = \min\{9, 11\} = 9;$

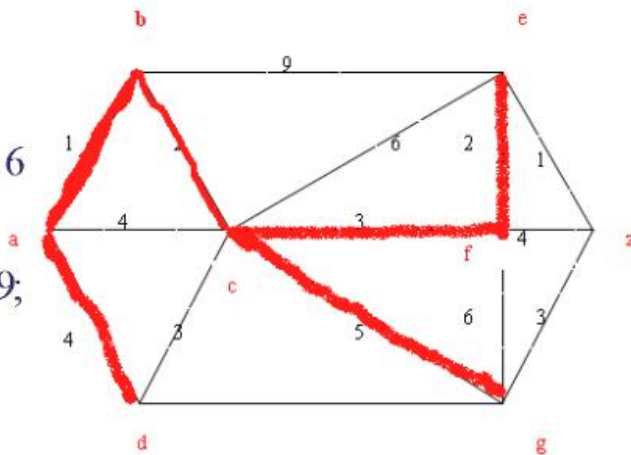
$prev[e] = f, prev[f] = prev[g] = c, prev[c] = b;$

$prev[b] = prev[d] = a, prev[z] = e$

此时没有道路被更新

并且当前 $\bar{S}$ 中最短的dist是 $dist[z]=9$ 则 $a \rightarrow z$ 的最短路已经确定即是 $a \rightarrow b \rightarrow c \rightarrow e \rightarrow f \rightarrow g$

接下来 $\bar{S}$ =空集, 迭代结束。



5. 图论与离散优化

• 5.2 最短路径问题

至此，从顶点a出发到各点的最短路已经全部求出：

$a \rightarrow a:0$

$a \rightarrow b:1$

$a \rightarrow b \rightarrow c:3$

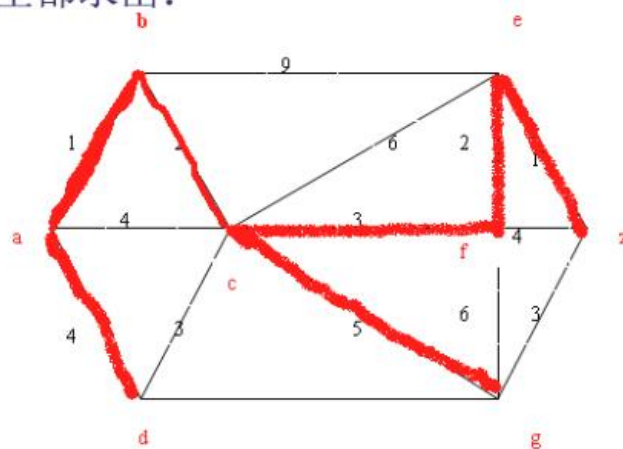
$a \rightarrow d:4$

$a \rightarrow b \rightarrow c \rightarrow f \rightarrow e:8$

$a \rightarrow b \rightarrow c \rightarrow f:6$

$a \rightarrow b \rightarrow c \rightarrow g:8$

$a \rightarrow b \rightarrow c \rightarrow f \rightarrow e \rightarrow z:9$



5. 图论与离散优化

• 5.2 最短路径问题

Dijkstra算法的分析

- 可终止性
 - 计数控制
- 正确性

需证明当算法终止时

 - $L(v) = d(s, v)$ 对一切 v 成立。
 - 由标记中的诸 u_i 确定的路径是一条最短路径
(这里 $d(s, v)$ 是 s 到 v 的最短路径长度, 即距离。)
- 复杂性
 - $O(n^2)$

